Infrastructure as Code

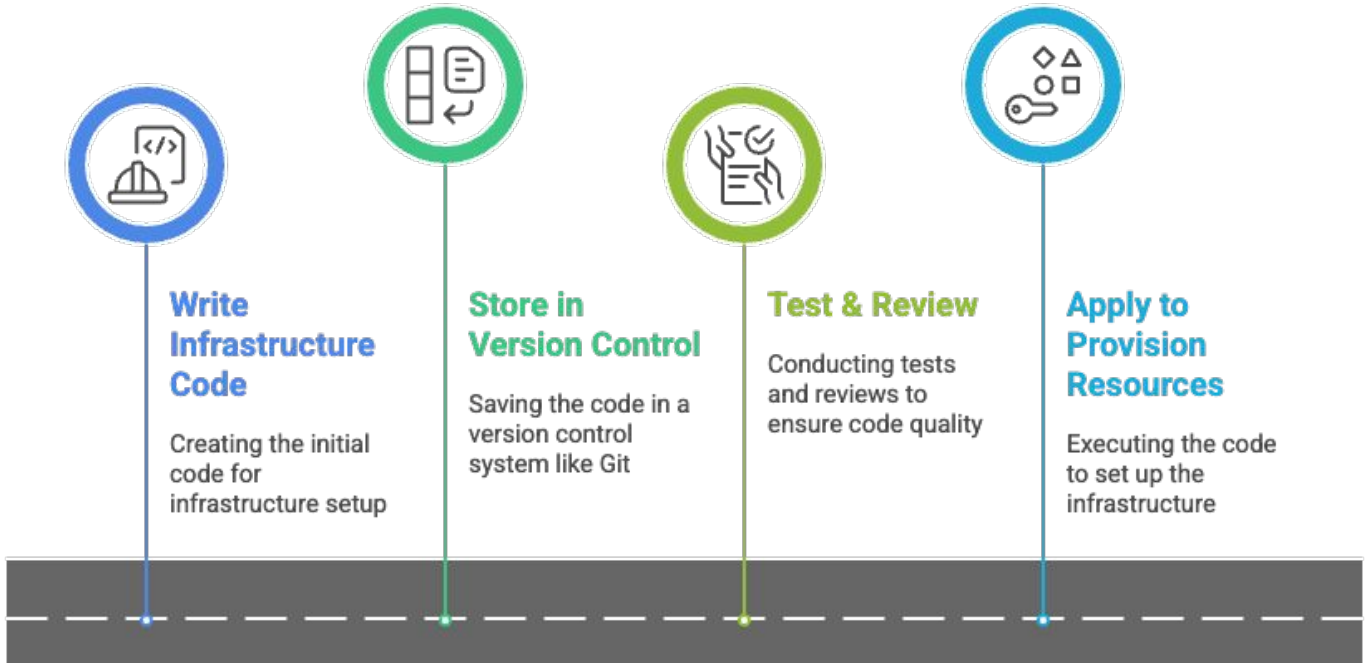
IaC

Terraform

IaC – Infrastructure as Code

- IaC is the practice of managing and provisioning infrastructure (servers, networks, databases, etc.) using code instead of manual processes.

Infrastructure as Code Workflow



Terraform

Terraform

Terraform

- Terraform is an open-source Infrastructure as Code (IaC) tool by HashiCorp.
- It allows you to define, provision, and manage infrastructure using declarative configuration files.

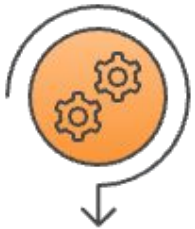
Key Points:

- **Cloud-agnostic** → Works with AWS, Azure, GCP, etc.
- **Declarative syntax** → Define *what* infrastructure should look like.
- **Immutable infrastructure** → Changes create new resources safely.
- **Version-controlled** → Infrastructure code stored in Git.

Terraform benefits

Multi-Cloud Support

Manage resources across multiple cloud providers.



Reusable Modules

Standardize infrastructure configurations by creating reusable modules.

State Management

Terraform keeps track of the status of your infrastructure resources.

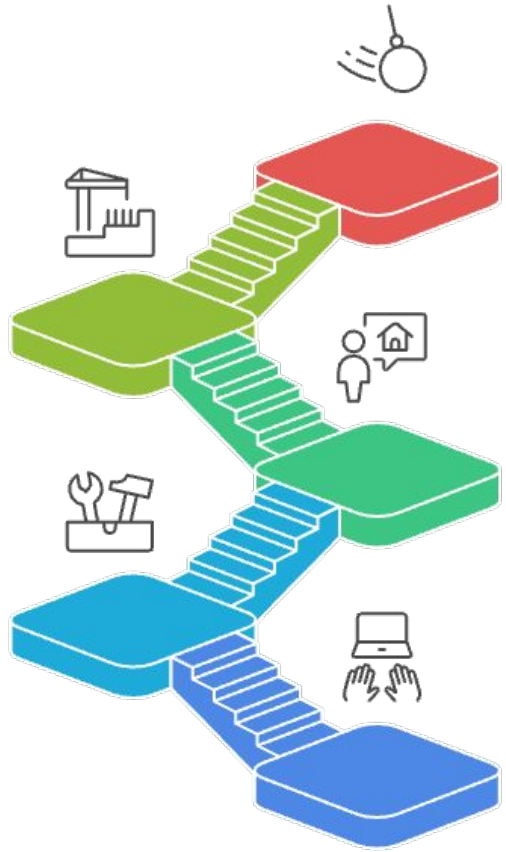
Automation & Scalability

Automate infrastructure deployments and easily scale resources up or down.

Collaboration

Teams can collaborate on the same infrastructure code using Terraform.

Terraform Workflow



Write

Define infrastructure in `.tf` files.

Init

Initialize working directory (`terraform init`).

Plan

Preview changes before applying (`terraform plan`).

Apply

Create/update resources (`terraform apply`).

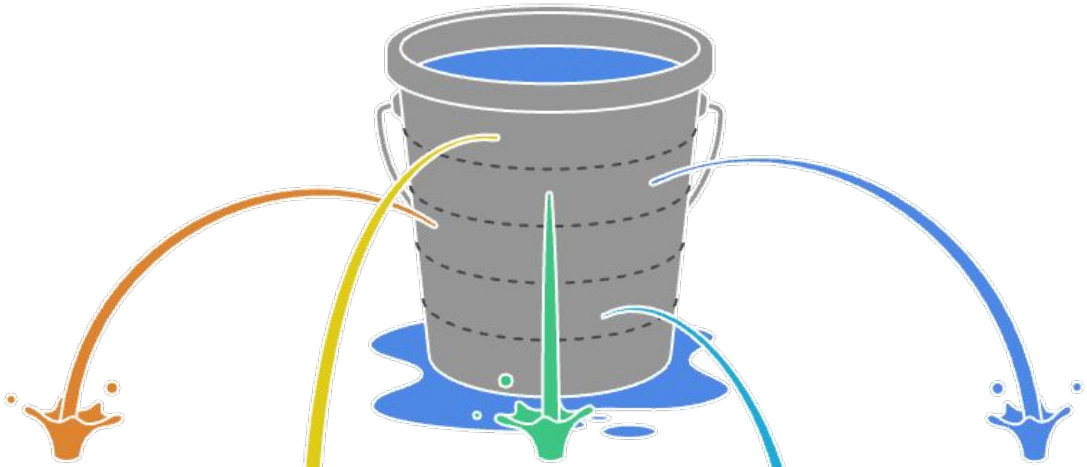
Destroy

Remove resources (`terraform destroy`).

Challenges in Traditional Infra

Terraform

Challenges in Traditional Infrastructure



Manual Provisioning

High human effort required

Error-Prone

Misconfigurations due to manual steps

Slow Deployment

Provisioning takes days or weeks

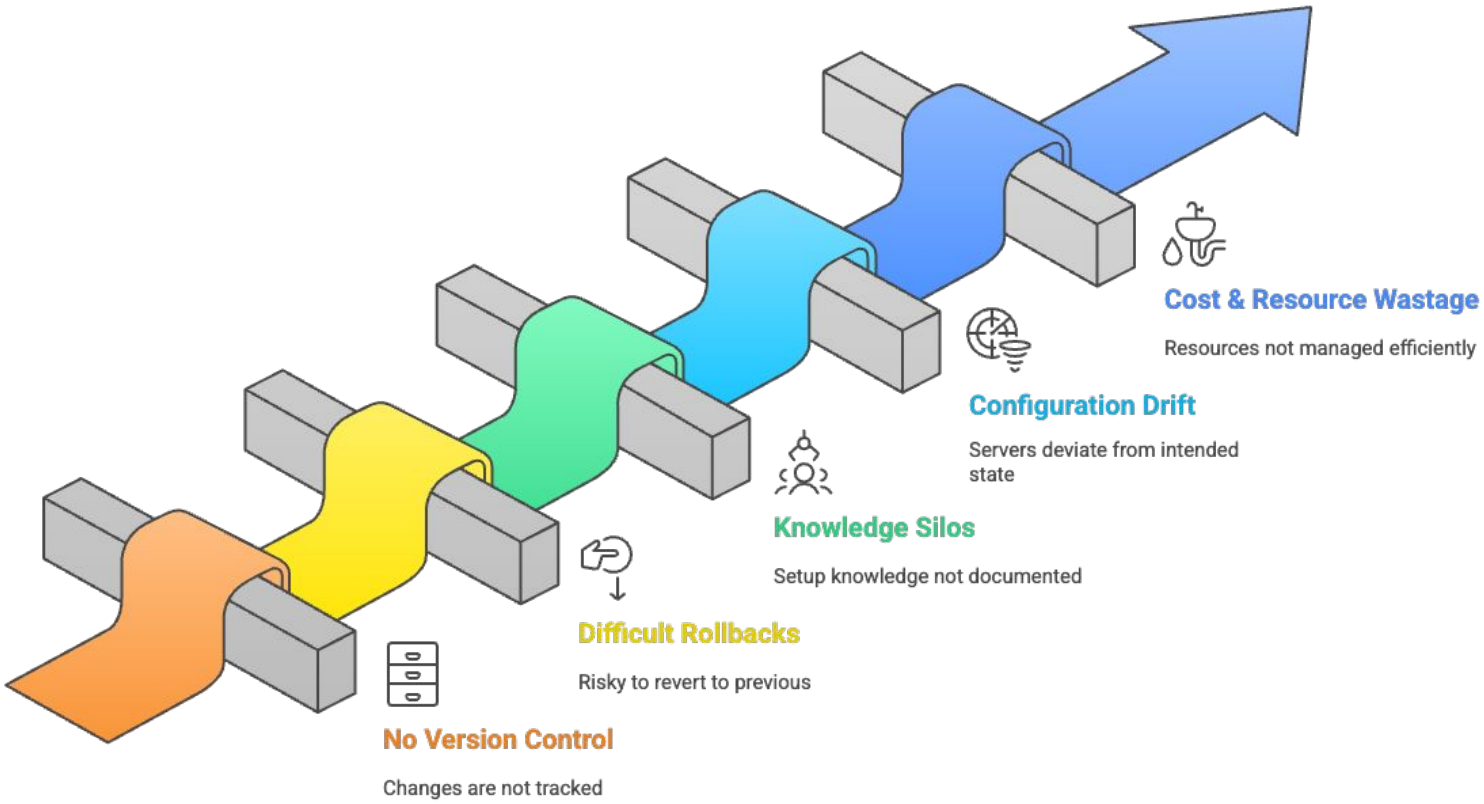
Lack of Consistency

Environment inconsistencies across teams

Poor Scalability

Difficult to scale during spikes

Limitations Before IaC

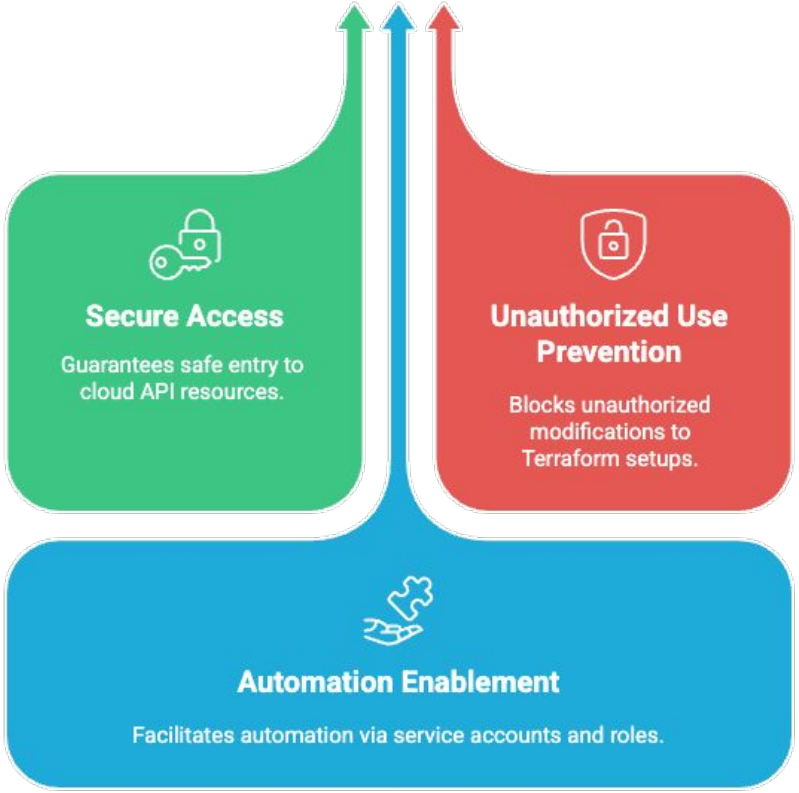


Authentication

Terraform

Authentication

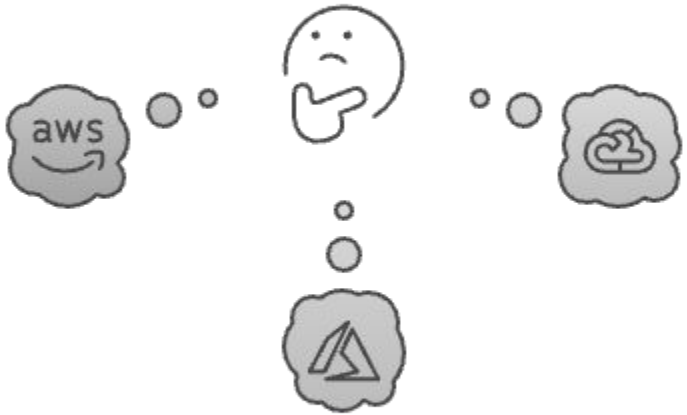
- Authentication means **verifying who Terraform is** before it can talk to a cloud provider like AWS, Azure, or GCP.



How to authenticate with cloud providers using Terraform?

AWS Authentication

Use environment variables, shared credentials file, AWS CLI login, or IAM role for EC2/ECS.



GCP Authentication

Use Application Default Credentials, service account key file, or gcloud CLI authentication.

Azure Authentication

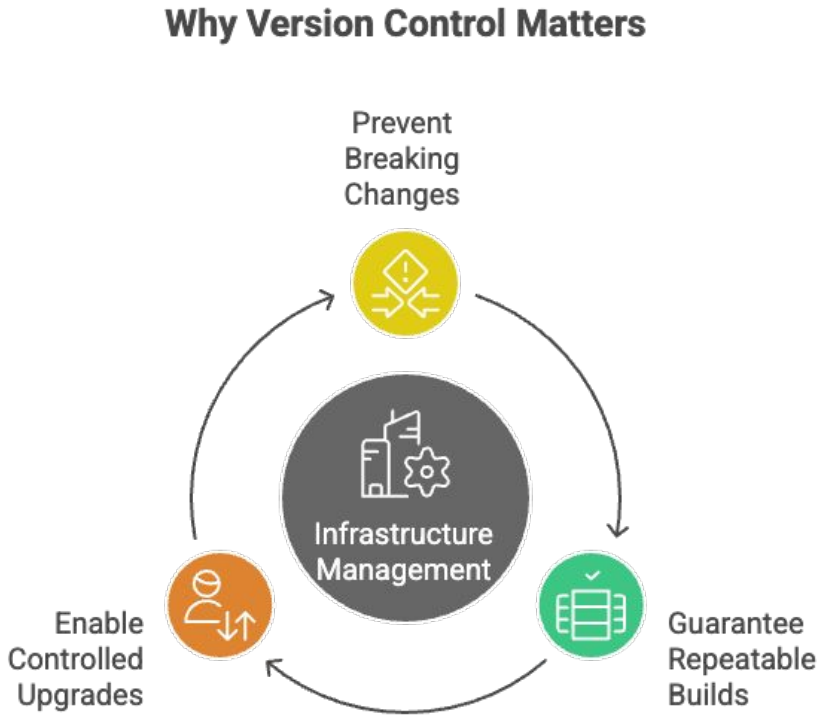
Authenticate via Azure CLI login, service principal, or managed identity.

Terraform Provider Versions

Terraform

Terraform Provider Version

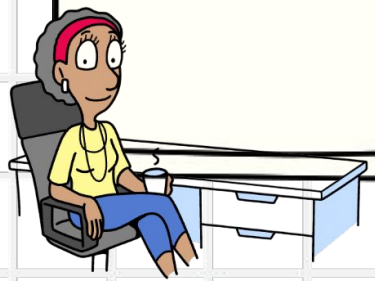
- To ensure Terraform uses a specific or compatible provider version for consistent deployments.



Operator	Meaning	Example	Explanation
=	Exact match	= 5.10.0	Only version 5.10.0
!=	Not equal to	!= 4.0.0	Any version except 4.0.0
> / <	Greater/Less than	> 5.0.0	Newer than 5.0.0
>= / <=	Greater/Less or equal	>= 4.0.0	Version 4.0.0 or newer
~>	Allows rightmost version component to increment	~> 5.0	Any version in the range 5.X

Provider

Terraform



Provider

- Plugins that allow Terraform to interact with cloud platforms, services, and APIs.
- **Role:** Translate Terraform configuration into API calls for resource creation, update, or deletion.
- **Examples:**
    a. Cloud → AWS, Azure, GCP
    b. Infrastructure → Kubernetes, VMware
    c. SaaS → GitHub, Datadog
- **Key Point:**

  Without providers, Terraform cannot manage infrastructure.

Setting Up Infrastructure with Terraform

3 **Use Resources**

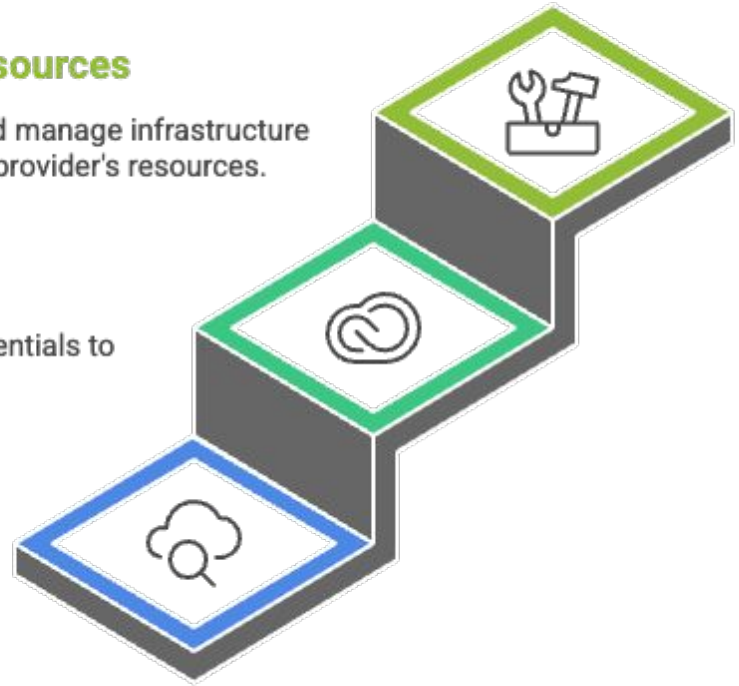
Create and manage infrastructure using the provider's resources.

2 **Authenticate**

Provide necessary credentials to access the platform.

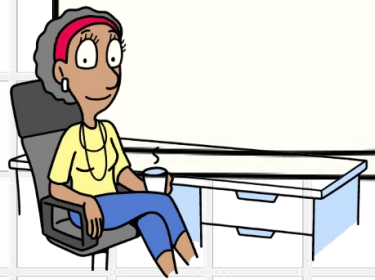
1 **Specify Provider**

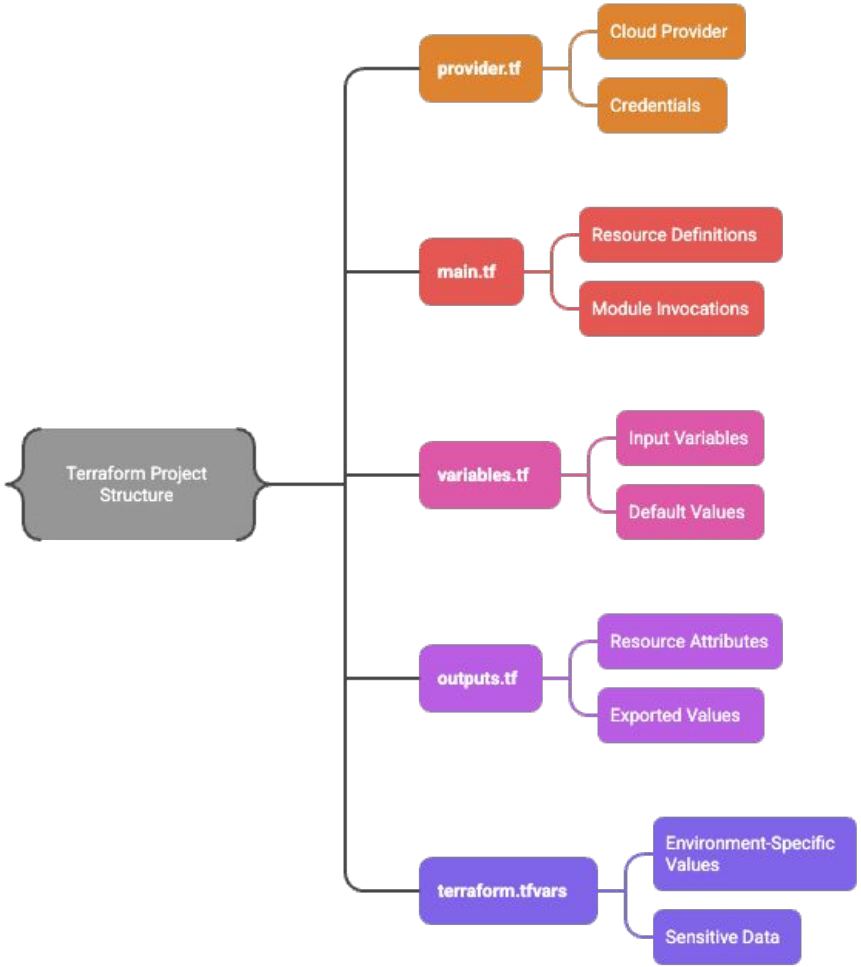
Choose the cloud platform to use with Terraform.



File Structure

Terraform



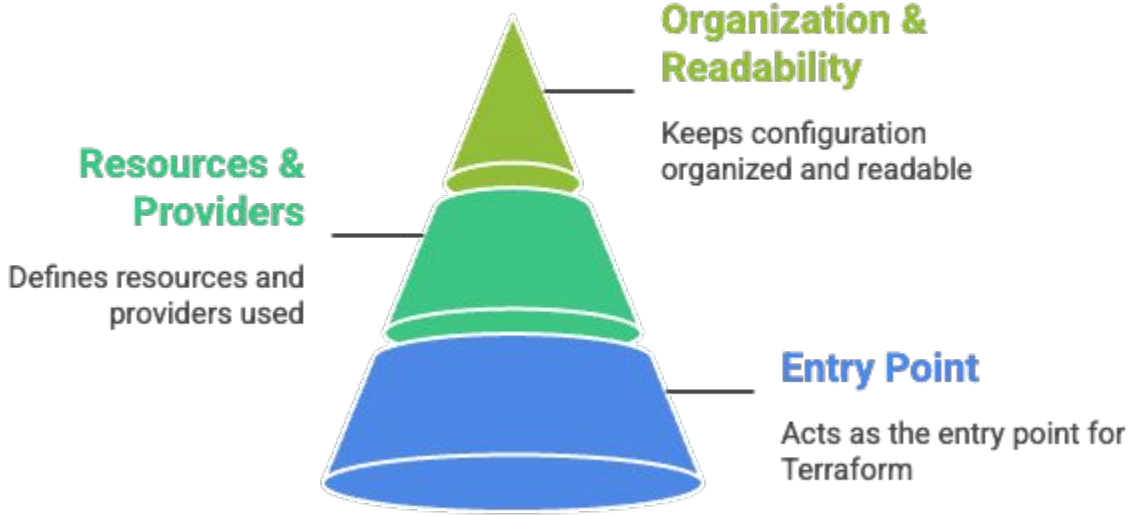

Terraform Project Structure
provider.tf
Cloud Provider
Credentials
main.tf
Resource Definitions
Module Invocations
variables.tf
Input Variables
Default Values
outputs.tf
Resource Attributes
Exported Values
terraform.tfvars
Environment-Specific Values
Sensitive Data

Step - 2 : main.tf

Terraform

main.tf

- main.tf is the **primary configuration file** in a Terraform project.
- It contains the **main infrastructure definitions** — the “what to build” part of your code.
- Terraform automatically loads all .tf files, but main.tf is typically used for the **core logic** of your setup.



Section	Purpose
terraform	Declares provider requirements and versions
provider	Configures how to connect to the provider
resource	Defines actual infrastructure components
data *(optional)*	Reads existing infrastructure
output *(optional)*	Displays useful information after deployment

main.tf > ...

```hcl
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_instance" "web" {
  ami           = "ami-0abcd1234"
  instance_type = "t2.micro"

  tags = {
    Name = "MyWebServer"
  }
}
```

main.tf > ...

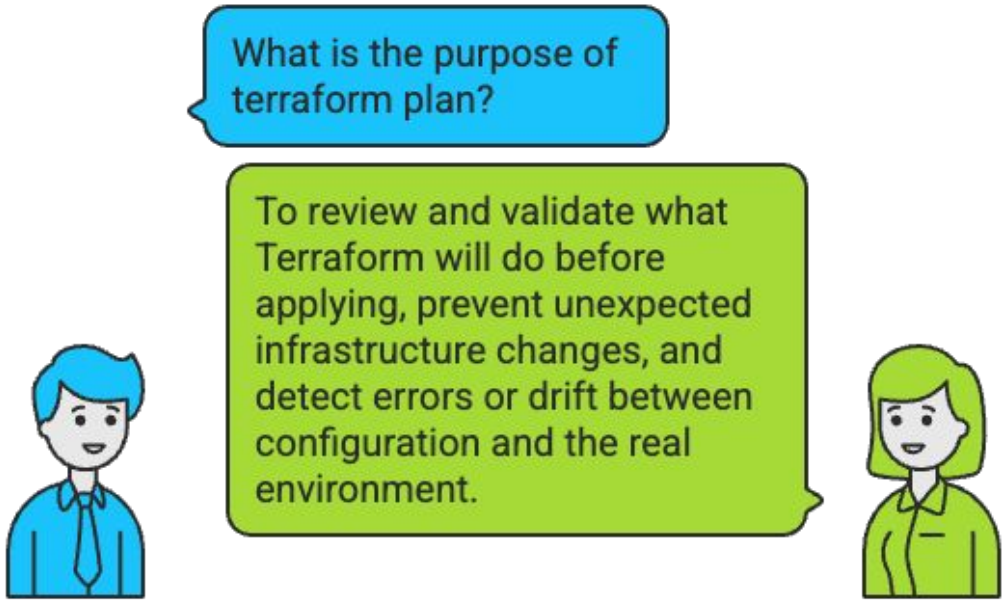
```hcl
resource "aws_instance" "web" {
  ami           = "ami-0abcd1234"
  instance_type = "t2.micro"

  tags = {
    Name = "MyWebServer"
  }
}
```

Step - 3 : terraform plan

Terraform

Terraform plan

- terraform plan is the **preview command** in Terraform.
- It shows what **actions** Terraform will take to reach the desired state *without making any changes*.



Key Points:

- Safe to run — **no resources are modified**.
- Compares current state with configuration files.
- Outputs an **execution plan** (add, change, destroy).

Key Takeaway:

“+” → Resource will be **created**.

“~” → Resource will be **modified**.

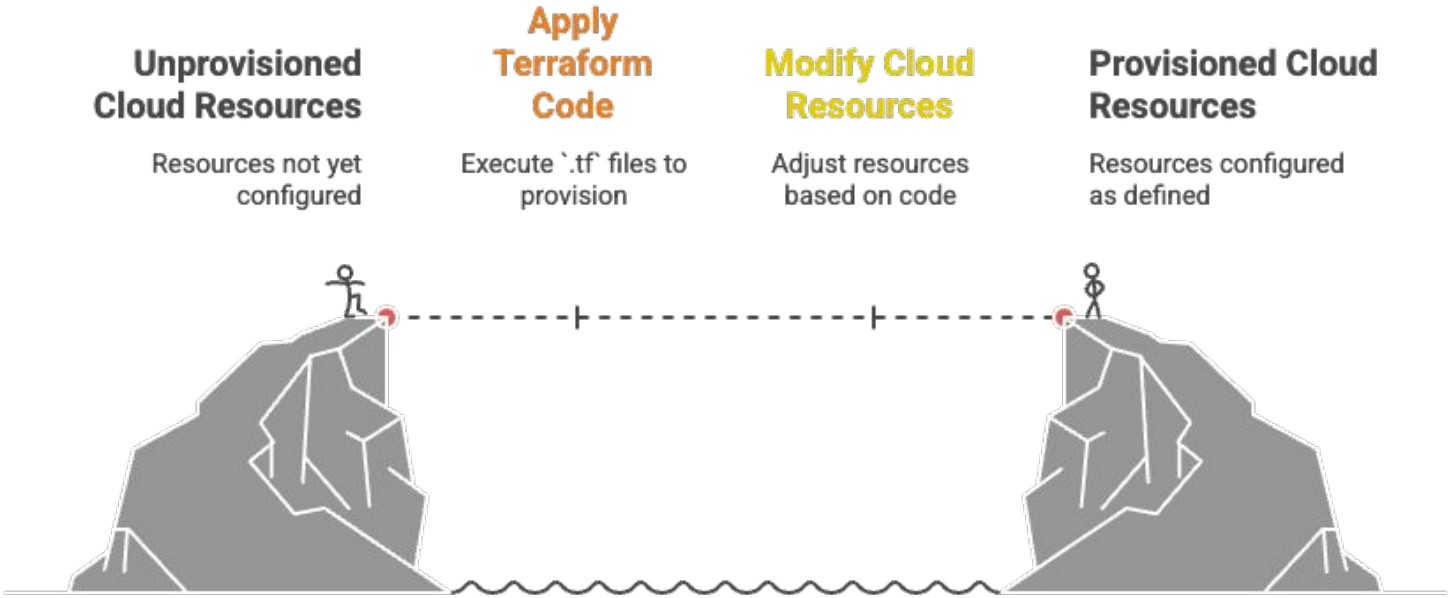
“–” → Resource will be **destroyed**.

Step - 4 : terraform apply

Terraform

Terraform apply

- terraform apply is the command that **executes** the changes proposed in terraform plan.
- It creates, updates, or deletes resources to make your infrastructure match the **desired state**.



Key Points:

- Executes the **plan** created by terraform plan.
- Prompts for **confirmation** before applying (unless -auto-approve is used).
- Updates the **state file** after changes are applied.

After Apply:

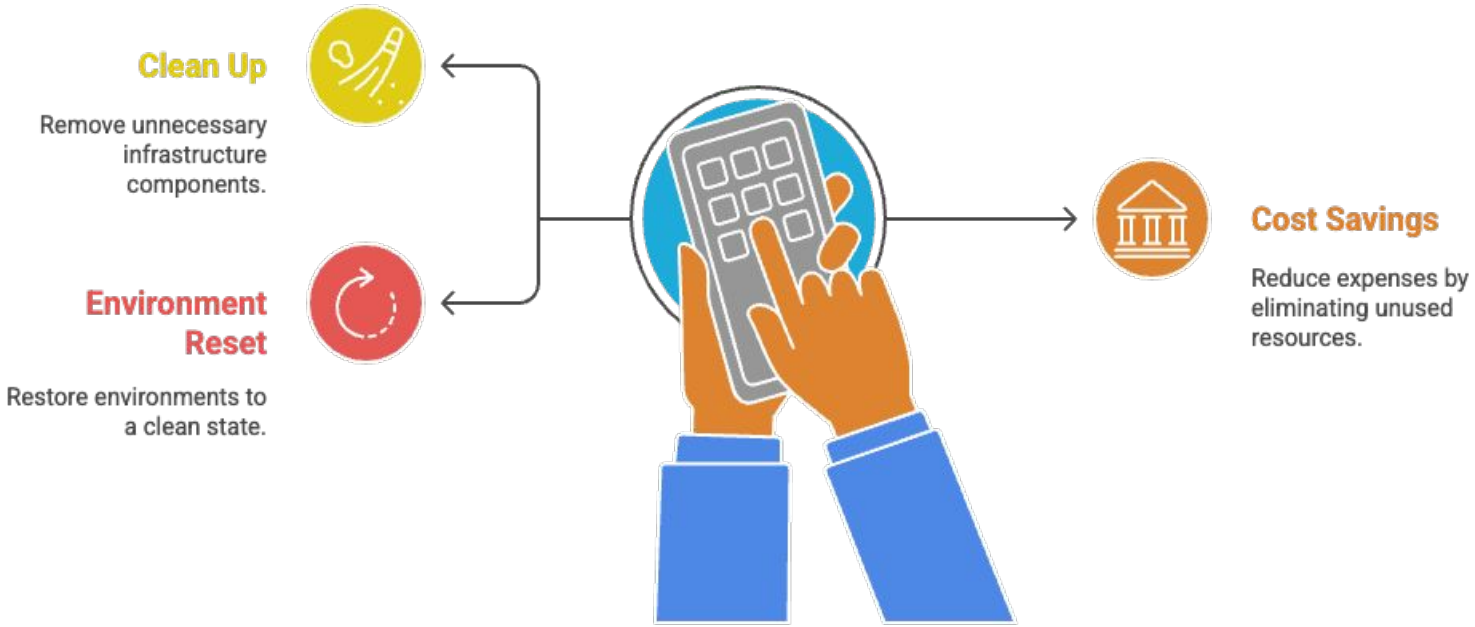
- Terraform updates the **terraform.tfstate** file.
- The new bucket now exists in AWS.
- Desired State = Current State

Step - 5 : terraform destroy

Terraform

Terraform destroy

- terraform destroy is the command used to **delete** all resources managed by Terraform.
- It safely removes infrastructure that was previously **created using Terraform**.



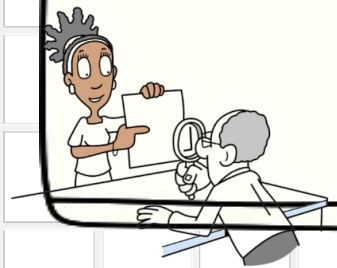
Key Points:

- Deletes all resources from the **current state**.
- Confirms before destruction unless `-auto-approve` is used.
- Updates the **terraform.tfstate** file after destruction.

After Destroy:

- Terraform **deletes** the resources.
- Updates the **state file** to remove destroyed resources.
- Returns the environment to a **clean slate**.

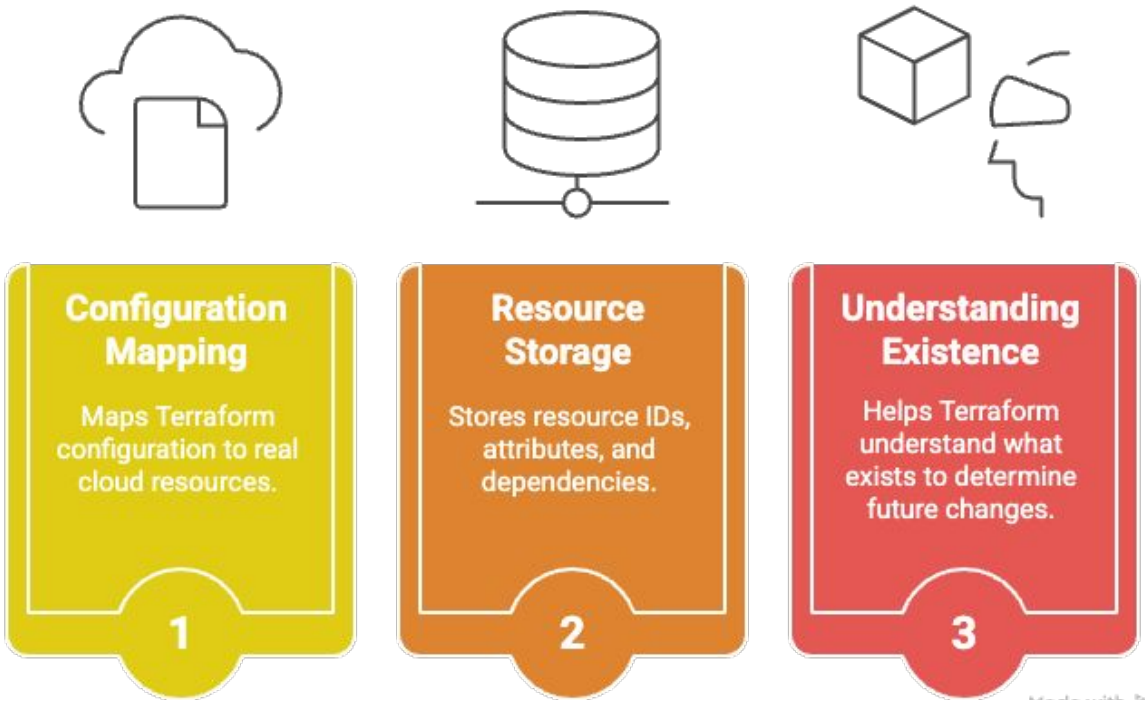
State File



Terraform

State File

- The **state file** (terraform.tfstate) is a **JSON file** where Terraform keeps track of all the **resources it manages**.



Key Points:

- Created automatically after the first terraform apply.
- Updated every time changes are made.
- Must be **kept secure**, as it can contain **sensitive data** (like passwords or ARNs).

NOTE:

- terraform.tfstate is critical for Terraform's accuracy.
- Store it securely.
- Never manually edit it.

Process:

1 Creation:

- Generated after running terraform apply and captures the **current state** of all resources.

2 Comparison:

- During terraform plan, Terraform compares:
    - **Current State** (from .tfstate)
    - **Desired State** (from .tf files)

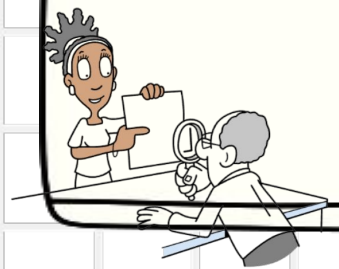
3 Update:

- When terraform apply runs, it updates the .tfstate file to reflect new changes.

4 Storage Options:

- **Local Backend:** terraform.tfstate stored locally (default).
- **Remote Backend:** Stored securely in cloud storage (e.g., S3, GCS, Terraform Cloud).

State File HandsOn



Terraform

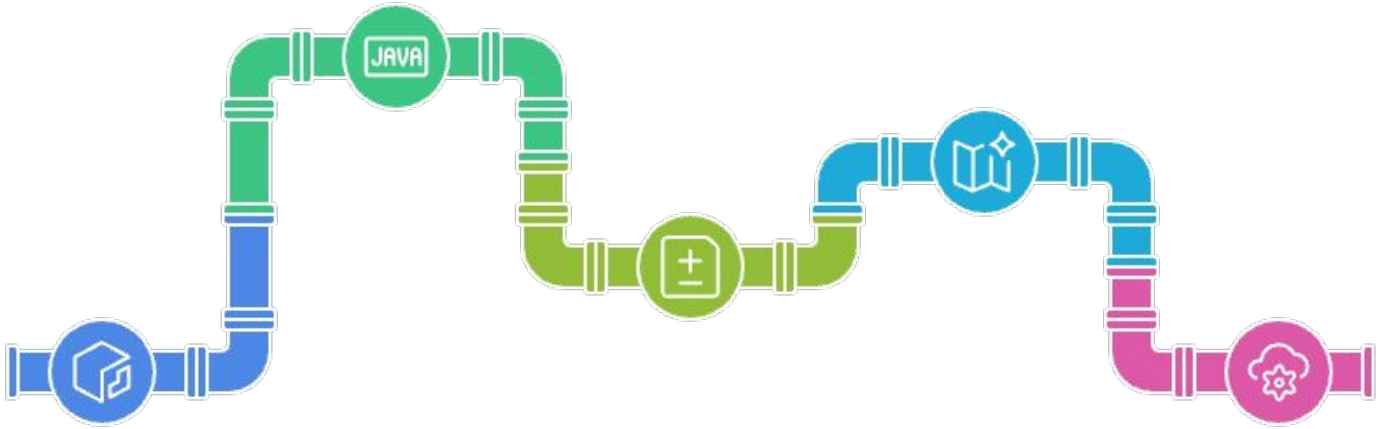
Desired State

vs

Current State

Terraform

Aspect	Current State	Desired State
Definition	Represents the *actual infrastructure* as recorded in the terraform.tfstate file.	Represents the *intended infrastructure* as described in Terraform configuration files (.tf files).
Source	Generated and updated by Terraform during previous runs.	Defined by the user through HCL configuration code.
Purpose	To track what resources currently exist and their properties.	To specify what infrastructure should exist.
Stored In	terraform.tfstate file (local or remote backend).	.tf files (e.g., main.tf, variables.tf, etc.).



01

Read Current State

Accessing the current infrastructure state from `terraform.tfstate`.

02

Parse Desired State

Analyzing configuration files to determine the desired infrastructure state.

03

Compare Both States

Identifying differences between the current and desired states.

04

Generate Execution Plan

Creating a plan using `terraform plan` to outline intended actions.

05

Apply Changes

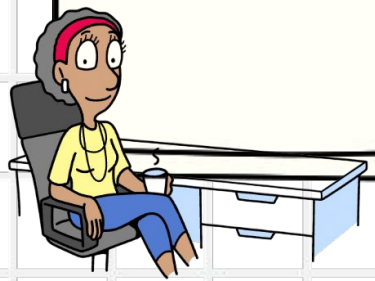
Updating the infrastructure and state file to match the desired configuration.

Desired vs Current State HandsOn

Terraform

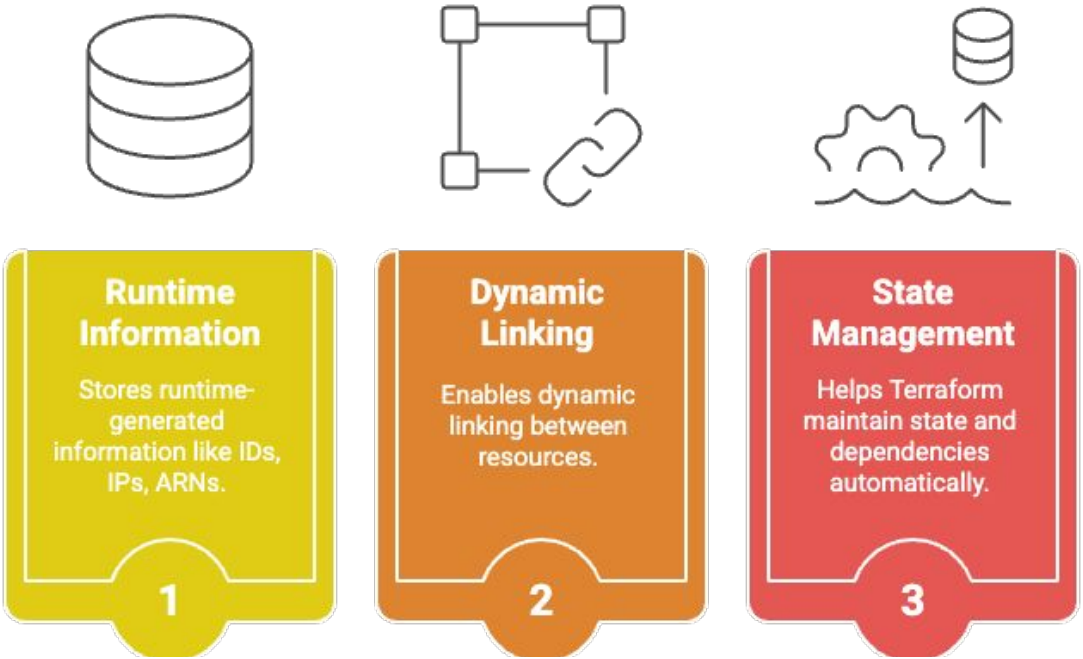
Attributes

Terraform



Attributes

- **Attributes** are values that **Terraform does not know until a resource is created or applied**.
- They are **automatically generated by the provider (e.g., AWS, Azure)**.



How Attributes Work:

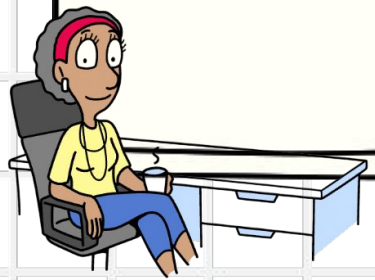
Stage	Terraform Action	Attribute Behavior
terraform plan	Terraform **cannot know** the final value yet	Shows as (known after apply)
terraform apply	Resource is created	Provider returns actual values
terraform state	State file stores the real computed values	Used for future dependency resolution

Syntax:

<RESOURCE_TYPE>.<RESOURCE_NAME>.<ATTRIBUTE>

Attributes HandsOn

Terraform



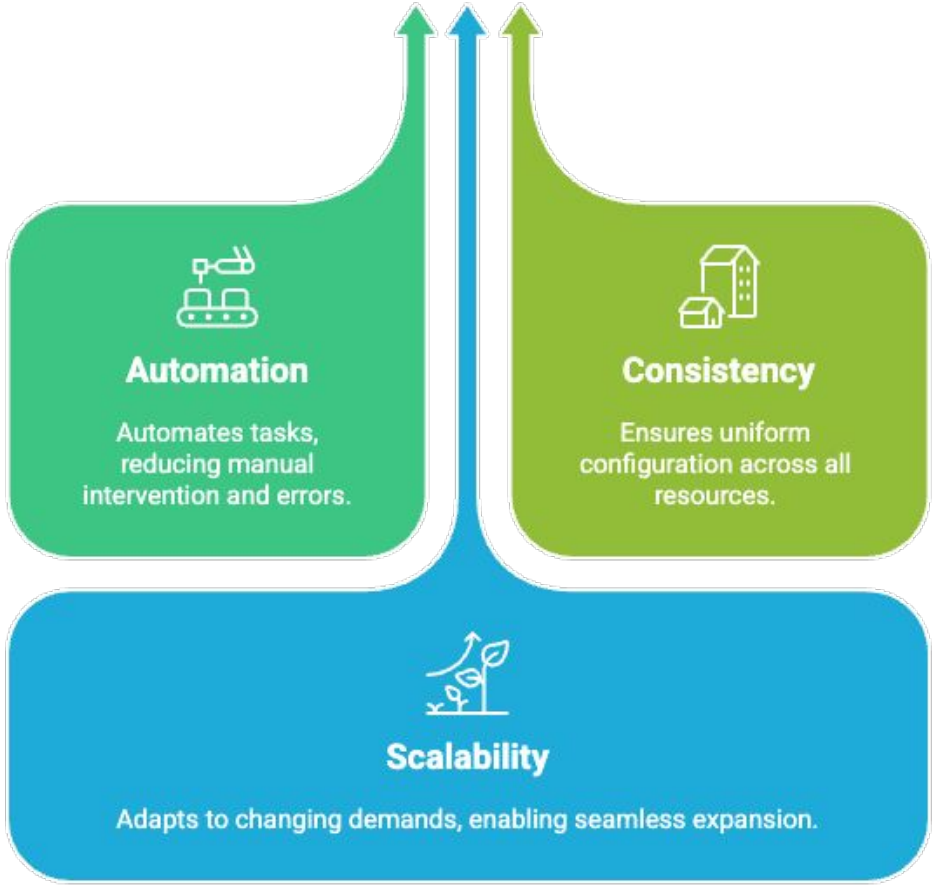
Attribute Cross Referencing

Terraform

Attributes - Cross Referencing

- Cross-referencing in Terraform means **using an attribute from one resource** in the configuration of **another resource**.
- Enables **dynamic linking** between resources without hardcoding values.
- Ensures **dependency management** — Terraform builds resources in the right order.

```
resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = "t2.micro"
}

resource "aws_eip" "elastic_ip" {
  instance = aws_instance.web.id  # Cross-reference
}
```



Terraform automatically detects:

- aws_eip depends on aws_instance

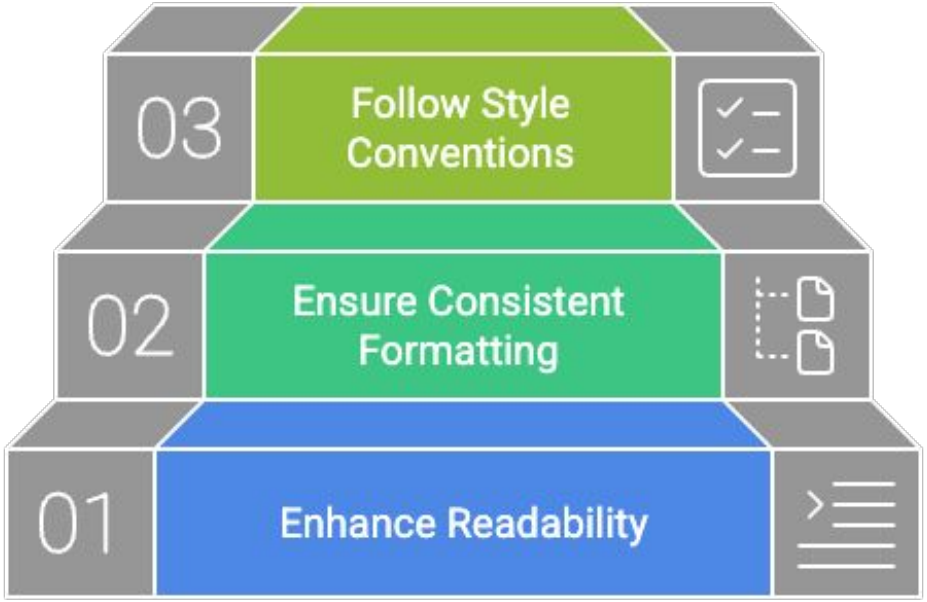
This ensures correct **resource creation order**.

Terraform fmt

Terraform

Terraform fmt

- terraform fmt is a **built-in Terraform command** that **formats configuration files** to a standard style.
- It ensures **readable, consistent, and clean code** across teams.



- **Basic Command:**

```
terraform fmt
```

- **Recursive Formatting:**

```
terraform fmt -recursive
```

- **Diff Output:**

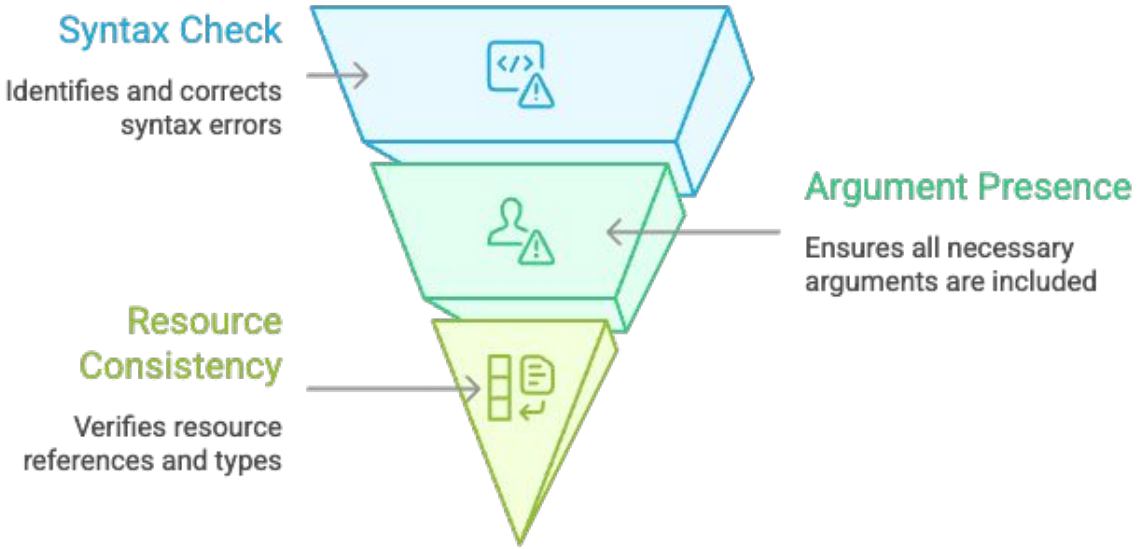
```
terraform fmt -diff
```

Terraform Validate

Terraform

Terraform validate

- terraform validate is a **Terraform CLI command** that checks whether the configuration files are **syntactically valid** and internally consistent.
- It **does not create or modify any resources**.



Basic Command:

```
terraform validate
```

Best Practices:

- Run terraform validate **before terraform plan and apply**
- Integrate into **CI/CD pipelines** to catch errors early
- Combine with terraform fmt for **code consistency and readability**
- Helps maintain **quality and reliability** in team projects

Resource Dependency

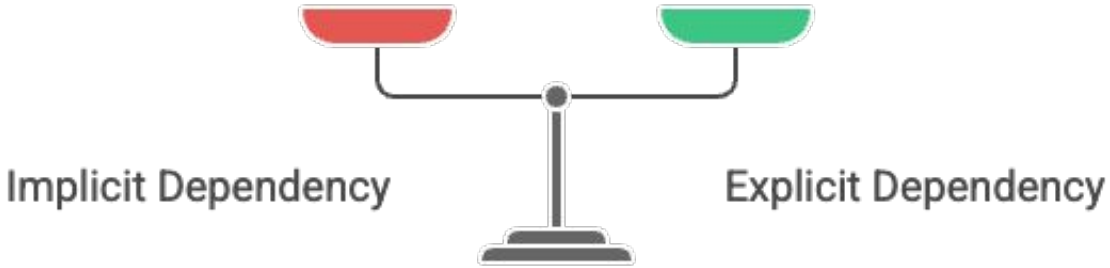
Terraform

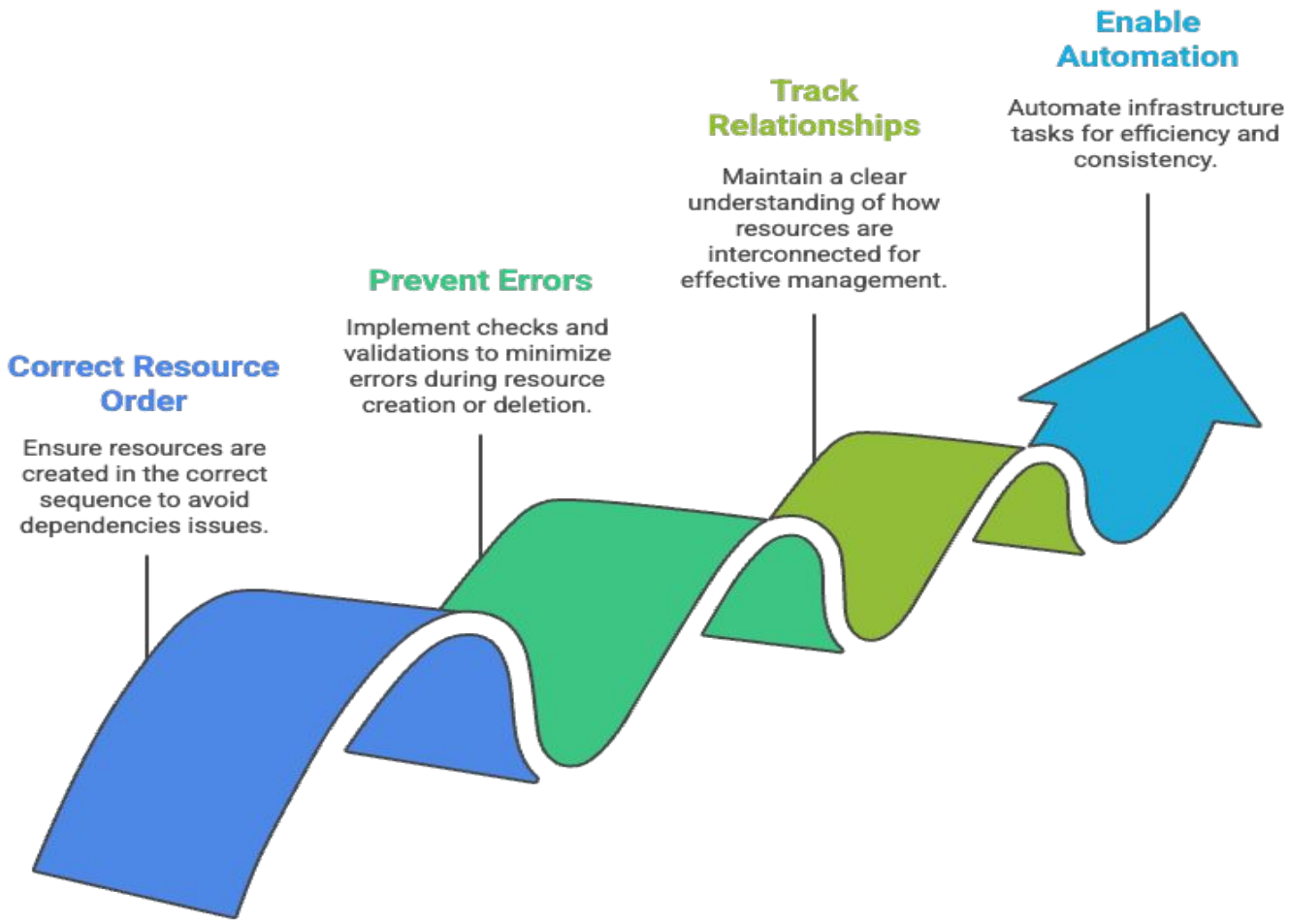
Resource Dependency

- **Resource dependencies** determine the **order** in which Terraform creates, updates, or destroys resources.

- Terraform builds a **dependency graph** automatically by analyzing **references** between resources.

Types of Resource Dependency:




Correct Resource Order
Ensure resources are created in the correct sequence to avoid dependencies issues.
Prevent Errors
Implement checks and validations to minimize errors during resource creation or deletion.
Track Relationships
Maintain a clear understanding of how resources are interconnected for effective management.
Enable Automation
Automate infrastructure tasks for efficiency and consistency.

Implicit Resource Dependency

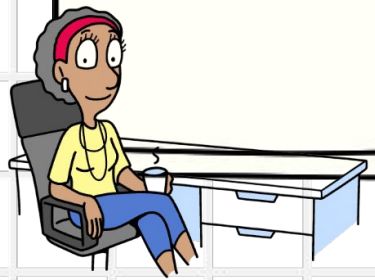
Terraform

Explicit Resource Dependency

Terraform

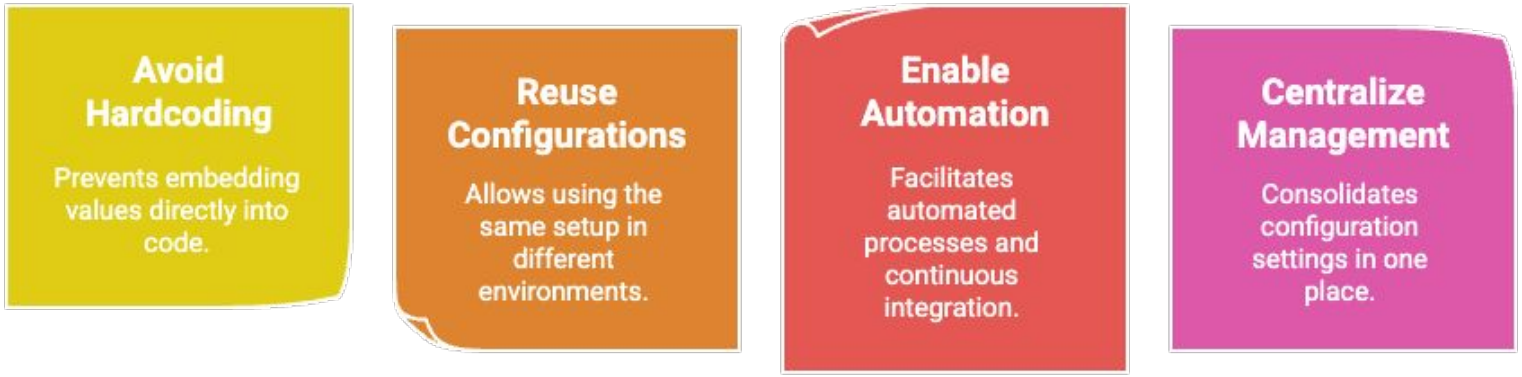
Variables

Terraform



Variables

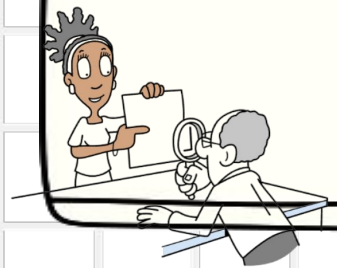
- Variables are **input values** used to **parameterize** Terraform configurations.
- They make your Terraform code **reusable, flexible, and easier to manage**.
- Think of variables as **placeholders** for values that may change between environments.



Variable Type	Description	Example Declaration
string	Represents a sequence of characters.	variable "region" { type = string }
number	Represents numeric values (integer or float).	variable "instance_count" { type = number }
bool	Represents true or false values.	variable "enable_logging" { type = bool }
list	Ordered collection of values of the same type.	variable "zones" { type = list(string) }
map	Collection of key-value pairs.	variable "tags" { type = map(string) }
set	Unordered collection of unique values.	variable "ips" { type = set(string) }
tuple	Ordered collection of values of different types.	variable "mix" { type = tuple([string, number, bool]) }

Method	Description	Example
Default value	Set a fallback value directly in the variable block in your .tf file.	variable "region" { default = "us-west-2" }
Environment variables	Set variables using shell environment variables prefixed with TF_VAR_. Useful for CI/CD pipelines or sensitive data.	export TF_VAR_region=us-east-1 Then run terraform apply.
terraform.tfvars file	Automatically loaded plain-text file in the root directory for variable assignments (HCL format).	In terraform.tfvars: region = "us-west-2"
.auto.tfvars files	Automatically loaded files matching the pattern *.auto.tfvars or *.auto.tfvars.json in the root directory.	In prod.auto.tfvars: region = "eu-central-1"
-var CLI flag	Directly set individual variable values on the command line.	terraform apply -var 'region=us-east-1'

Variable Block



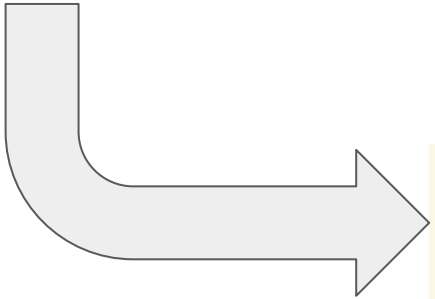
Terraform

Variable Block

- A **default value** is assigned to a variable if **no value is explicitly provided**.
- It ensures Terraform can run without requiring user input every time.

Behavior	Description
Default Value	Used when no other input is provided
Overriding	Higher-precedence inputs replace defaults
Purpose	Simplifies configuration and testing

```hcl
variable "instance_type" {
  type    = string
  default = "t2.micro"
  description = "EC2 instance type"
}
```



```hcl
resource "aws_instance" "example" {
  ami           = "ami-123456"
  instance_type = var.instance_type
}
```

Environment Variable

Terraform

Environment Variable

- Environment variables allow you to **set variable values outside** your Terraform configuration files.
- They are especially useful for **sensitive data** (like credentials) or **dynamic configurations** in CI/CD pipelines.

Feature	Benefit
Security	Keeps sensitive values (like keys) out of .tf files
Portability	Works well across local and CI/CD environments

```
export TF_VAR_instance_type="t2.large"
```

terraform.tfvars

Terraform

terraform.tfvars

- A **terraform.tfvars** file is used to **define variable values separately** from the main configuration.
- It helps you **organize and reuse** variable values without editing the main .tf files.

terraform.tfvars > ...

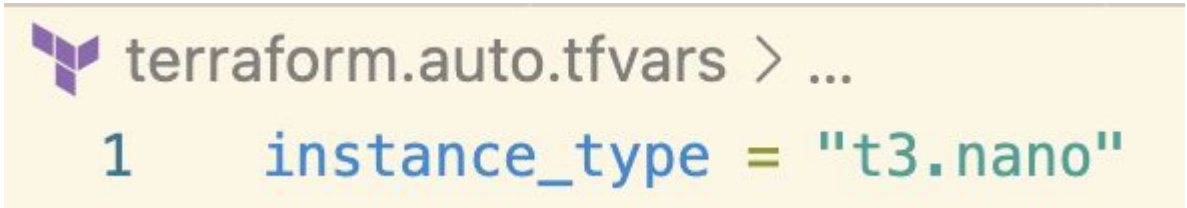
```
1    instance_type = "t3.small"
```

terraform.auto.tfvars

Terraform

terraform.auto.tfvars

- An **auto.tfvars** file is a **special variable definition file** that Terraform **automatically loads** during execution — no need to specify it using -var-file.



-var CLI flag

Terraform

-var CLI flag

- The -var flag lets you **pass variables directly from the command line** during plan or apply.
- Useful for **quick overrides** or **temporary variable changes**.
- It has **highest precedence** — overrides default, .tfvars, and .auto.tfvars.

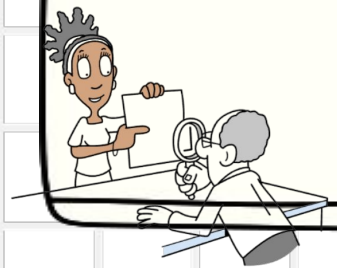
```
terraform plan -var="instance_type=t2.xlarge"
```

CleanUp

Terraform

Data Types

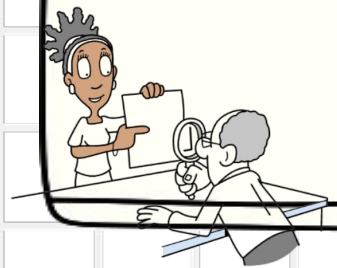
String, Number, Bool



Terraform

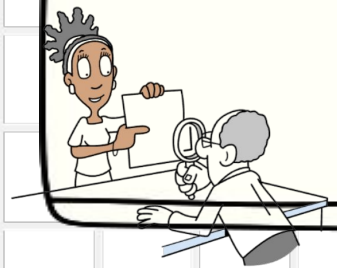
Best Practice

Terraform



Data Type

List



Terraform

Data Type – List

- A **list** is an **ordered collection** of values of the **same data type** (e.g., strings, numbers).
- Used to define multiple similar values such as regions, availability zones, or instance types.

Declaration:

```
variable "instance_types" {
  type    = list(string)
  default = ["t2.micro", "t2.small", "t2.nano"]
}
```

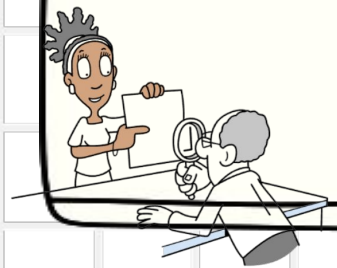
```
variable "instance_types" {
  type    = list(string)
  default = ["t2.micro", "t2.micro", "t2.micro"]
}
```

Usage:

```
resource "aws_instance" "server1" {
  instance_type = var.instance_types[0]
  ami = "ami-07860a2d7eb515d9a"
}
```

Data Type

Map



Terraform

Data Type – Map

- A **map** is a **collection of key–value pairs**.
- Keys must be **unique**.
- Values must be of the **same type** (e.g., all strings, all numbers).

Declaration:

```
variable "instance_types" {
  type = map(string)
  default = {
    dev   = "t2.micro"
    stage = "t2.small"
    prod  = "t2.large"
  }
}
```

Usage:

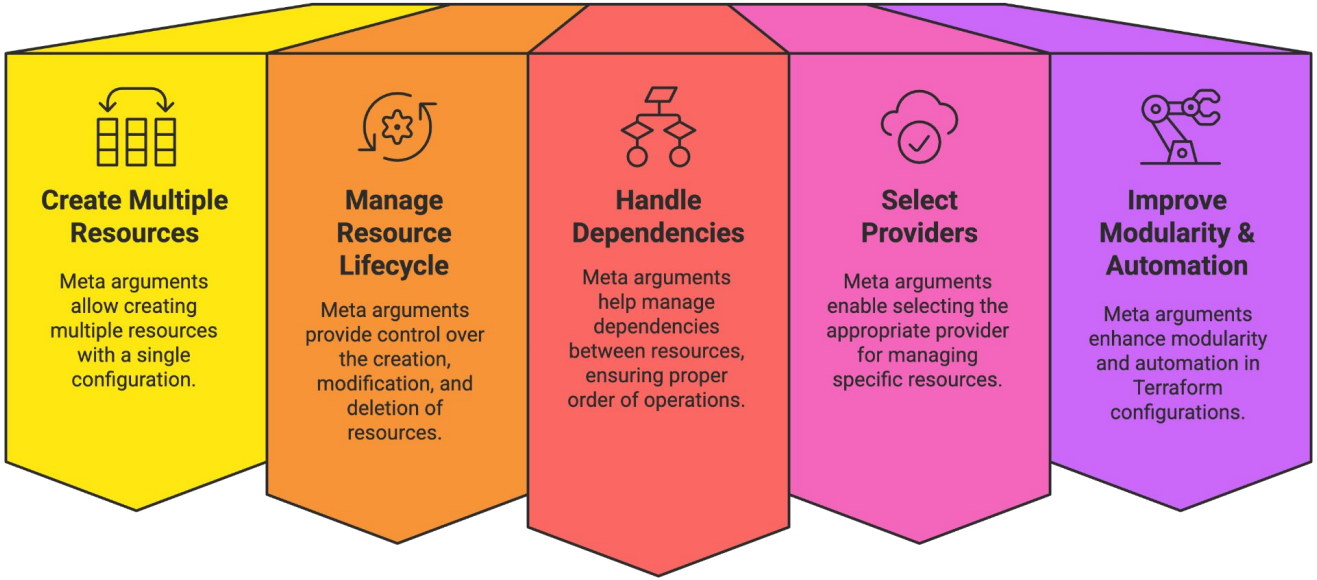
```
resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = var.instance_types["prod"]
}
```

Meta Arguments

Terraform

Meta-Arguments

- Meta-arguments are **special keywords** that can be used with **any resource or module** in Terraform.
- They control **how resources behave**, not what they contain.

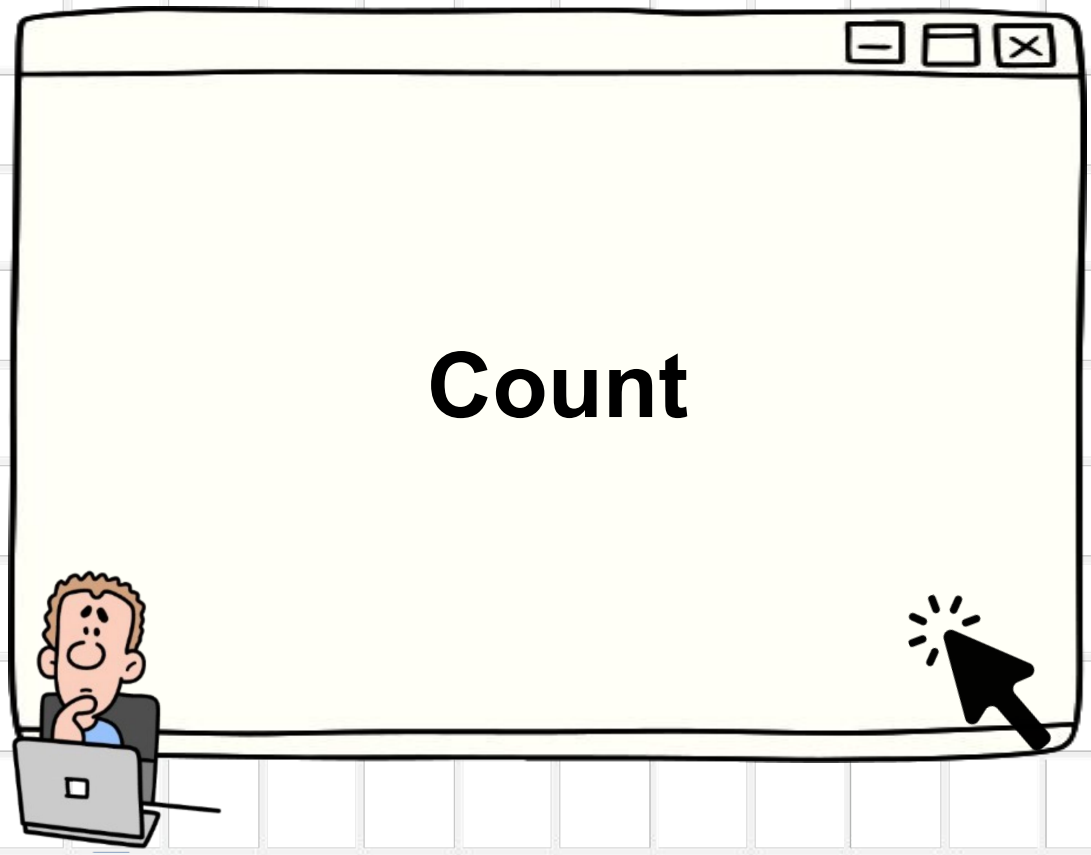


Common Meta-Arguments:

Meta-Argument	Purpose	Example
count	Create multiple instances of a resource	count = 3
for_each	Iterate over set/map and create resources dynamically	for_each = var.subnets
depends_on	Add **explicit dependencies**	depends_on = [aws_vpc.main]
lifecycle	Protect, replace, or ignore changes	lifecycle { prevent_destroy = true }
provider	Use a **specific provider configuration**	provider = aws.us_east

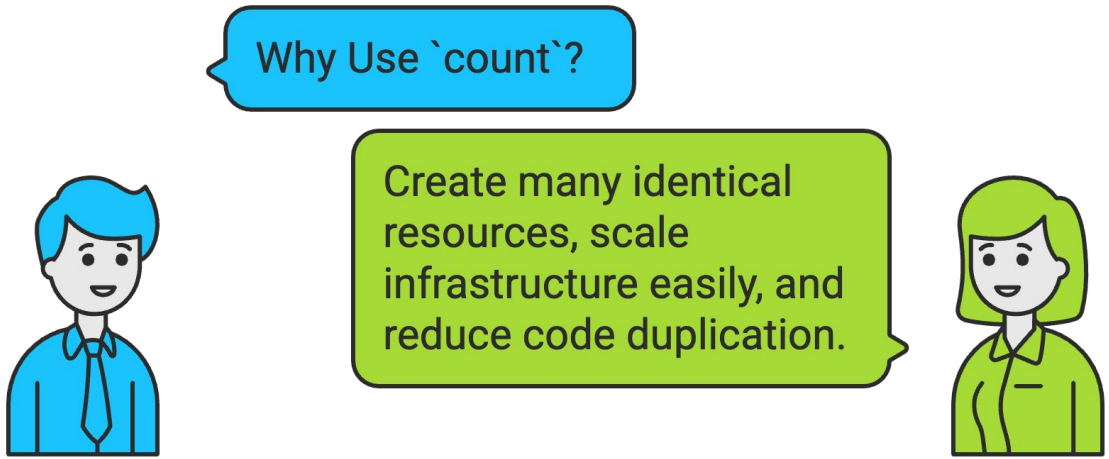
Count

Terraform



Count

- count is a Terraform **meta-argument** that lets you create **multiple instances** of a resource using a **single resource block**.
- Works like a loop — repeats the resource based on the number given.



Usage:

```
resource "aws_instance" "web" {
  count         = 3
  ami           = "ami-123456"
  instance_type = "t2.micro"
}
```

```
resource "aws_instance" "web_servers" {
  count = 3
  ami           = "ami-07860a2d7eb515d9a"
  instance_type = "t2.micro"

  tags = {
    Name = "web-server-${count.index}"
  }
}
```

for_each

Terraform

for_each

- for_each is a meta-argument used to **create multiple resources** based on a **map or set of strings**.
- Each resource gets a **unique key** instead of a numeric index.

Usage:

Using Set:

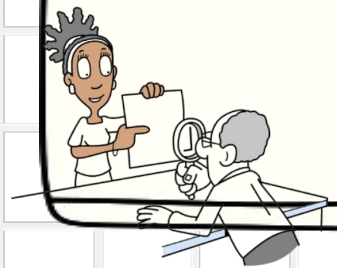
```
variable "buckets" {
  type = set(string)
  default = ["app", "logs", "backup"]
}

resource "aws_s3_bucket" "bucket" {
  for_each = var.buckets
  bucket   = "my-${each.value}-bucket"
}
```

Using Map:

```
variable "instances" {
  type = map(string)
  default = {
    web  = "t2.micro"
    db   = "t3.small"
  }
}

resource "aws_instance" "servers" {
  for_each      = var.instances
  ami           = "ami-123456"
  instance_type = each.value
  tags = {
    Name = each.key
  }
}
```

Data Type

Set



Terraform

Data Type – Set

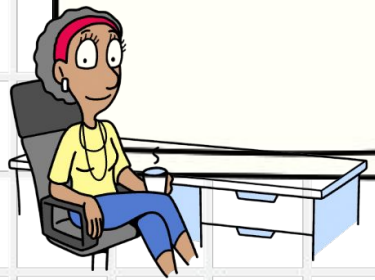
- A **set** is an **unordered collection of unique values**.
- Duplicate values are **automatically removed**.
- All values in a set must be of the **same type** (e.g., all strings).

Usage:

```
variable "buckets" {
  type = set(string)
  default = ["app", "logs", "backup"]
}

resource "aws_s3_bucket" "bucket" {
  for_each = var.buckets
  bucket   = "my-${each.value}-bucket"
}
```

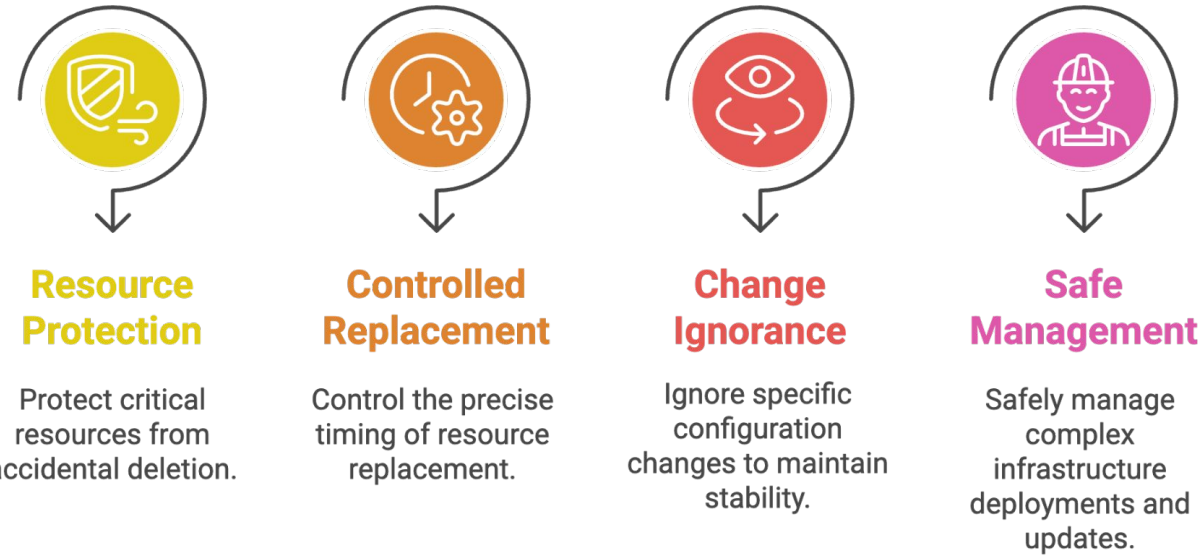
Lifecycle

Terraform



Lifecycle

- A **lifecycle** block is a Terraform meta-argument that controls **how** a resource behaves during creation, update, and deletion — not what it contains.



Types:

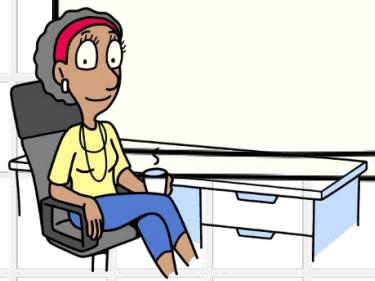
Meta-Argument	Purpose
create_before_destroy	Create new resource *before* destroying the old one
prevent_destroy	Blocks accidental deletion
ignore_changes	Ignores updates to specified attributes

Create before destroy

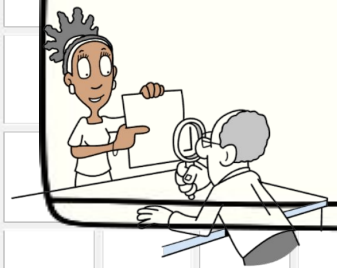
Terraform

Prevent destroy

Terraform



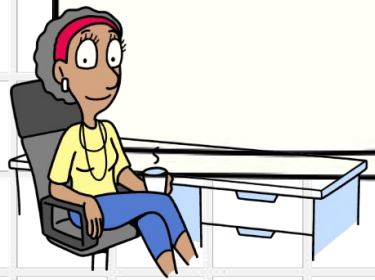
Ignore Changes



Terraform

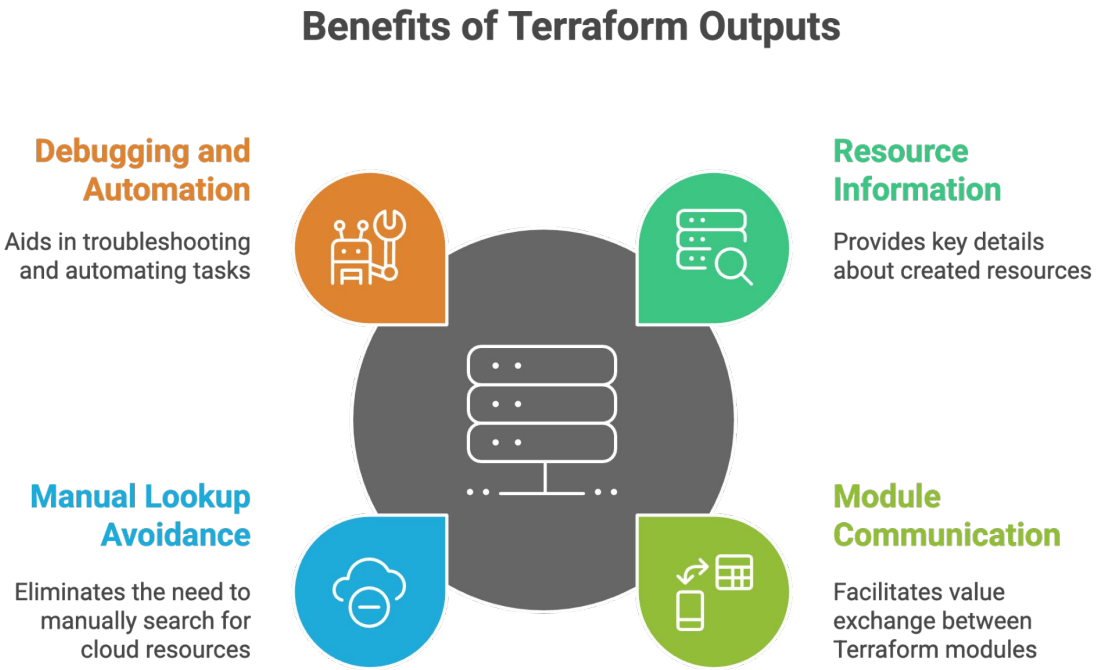
Output

Terraform



Output

- Outputs are **values returned by Terraform** after a successful apply.
- They help you **view important resource attributes**, such as IDs, IP addresses, ARN, etc.



Usage:

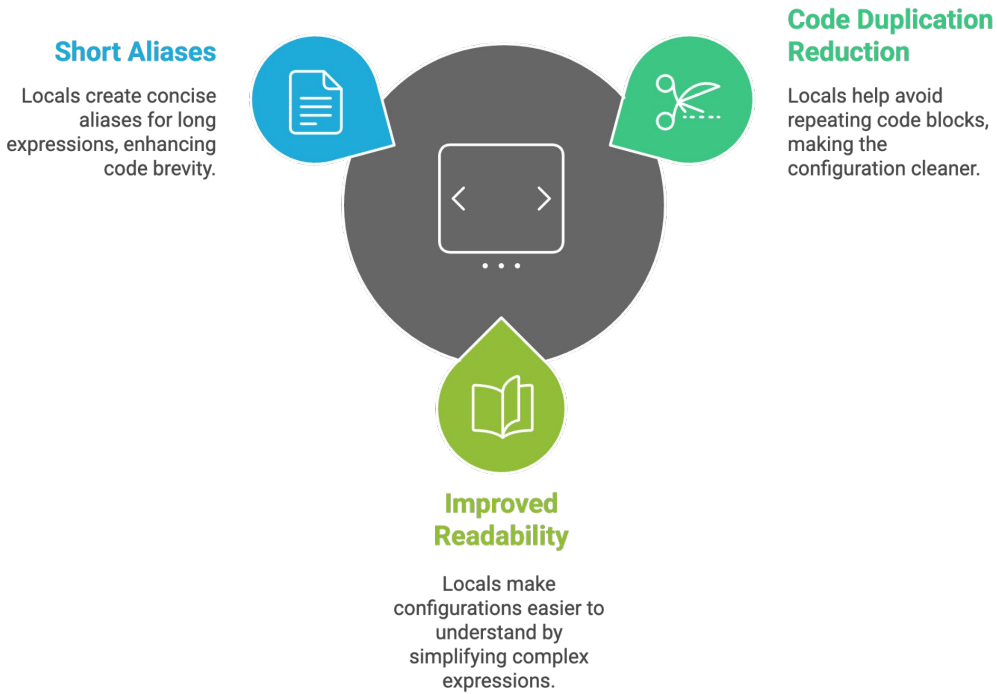
```
resource "aws_instance" "web" {
  ami           = "ami-123456"
  instance_type = "t2.micro"
}

output "public_ip" {
  value = aws_instance.web.public_ip
}
```

Locals

Terraform

Locals

- locals allow you to define **local named values** inside a Terraform configuration.
- Used to **avoid repeating values**, simplify expressions, and improve readability.



Usage:

```
locals {
  project_name = "my-app"
  environment  = "production"
}

resource "aws_s3_bucket" "data" {
  bucket = "terraform-data-bucket-2025"

  tags = {
    Project     = local.project_name
    Environment = local.environment
  }
}
```

Locals HandsOn

Terraform

Data Sources

Terraform

Data Sources

- Data sources allow Terraform to **fetch information about existing resources** from a provider (AWS, Azure, GCP, etc.).
- They are **read-only** — Terraform **does not create or modify** these resources.
- Used when infrastructure already exists and you need to reference it.

Best Practices

✔ Use data sources for **existing infra only**

✔ Use filters to narrow results

✔ Don’t try to modify resources retrieved via data sources

✔ Use outputs to expose helpful values

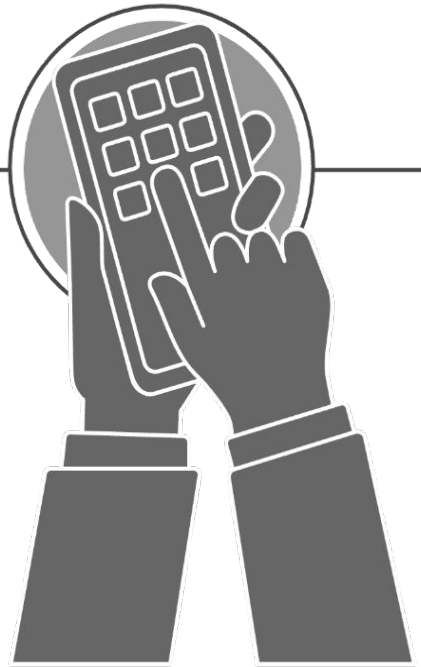
Benefits of Data Sources

Avoid Hardcoding

Data sources prevent hardcoding IDs, ARNs, and names.

Dynamic Configurations

Data sources make configurations dynamic and environment-agnostic.



Reuse Resources

Data sources allow reusing existing cloud resources.

Reduce Manual Lookups

Data sources reduce manual lookups in the cloud console.

Usage:

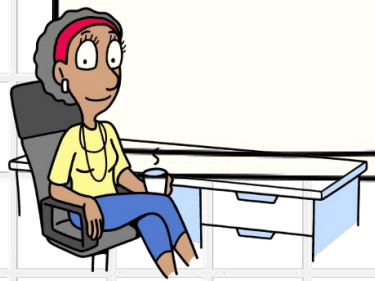
```
data "aws_region" "current" {}

output "my_region" {
  value = "I am running in ${data.aws_region.current.name}"
}
```

```
data "aws_s3_bucket" "my_bucket" {
  bucket = "my-existing-bucket-name"
}

output "bucket_info" {
  value = "Bucket ${data.aws_s3_bucket.my_bucket.id} is in ${data.aws_s3_bucket.my_bucket.region}"
}
```

Data Sources HandsOn (Clean Code)

Terraform

Conditions



Terraform

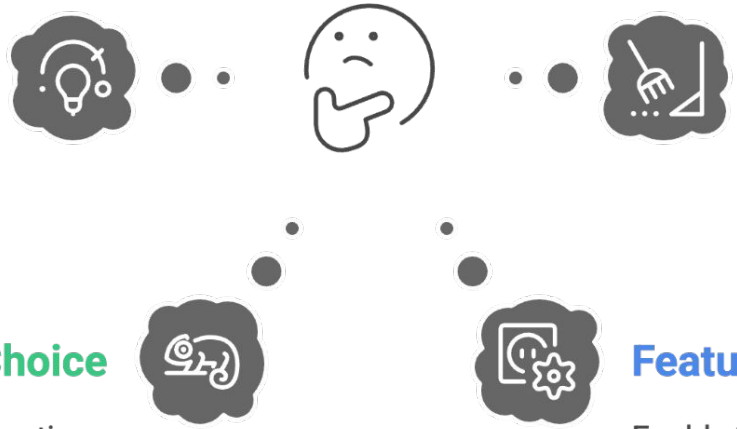
Conditions

- Conditions in Terraform allow you to **execute logic** based on whether an expression is **true or false** using the **ternary operator**:

Syntax:

```
condition ? value_if_true : value_if_false
```

Why use conditions in Terraform?



Resource Creation

Create resources only when needed, optimizing resource usage.

Avoid Duplication

Avoid duplication and hardcoding, maintaining code cleanliness.

Configuration Choice

Choose different configurations per environment, ensuring flexibility.

Feature Enablement

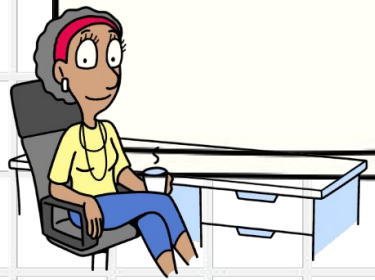
Enable/disable features dynamically, enhancing adaptability.

Usage:

```
variable "environment" {
  default = "dev"
}

resource "aws_instance" "web" {
  ami           = "ami-12345"
  instance_type = var.environment == "prod" ? "t2.large" : "t2.micro"
}
```

Conditions HandsOn

Terraform



Functions

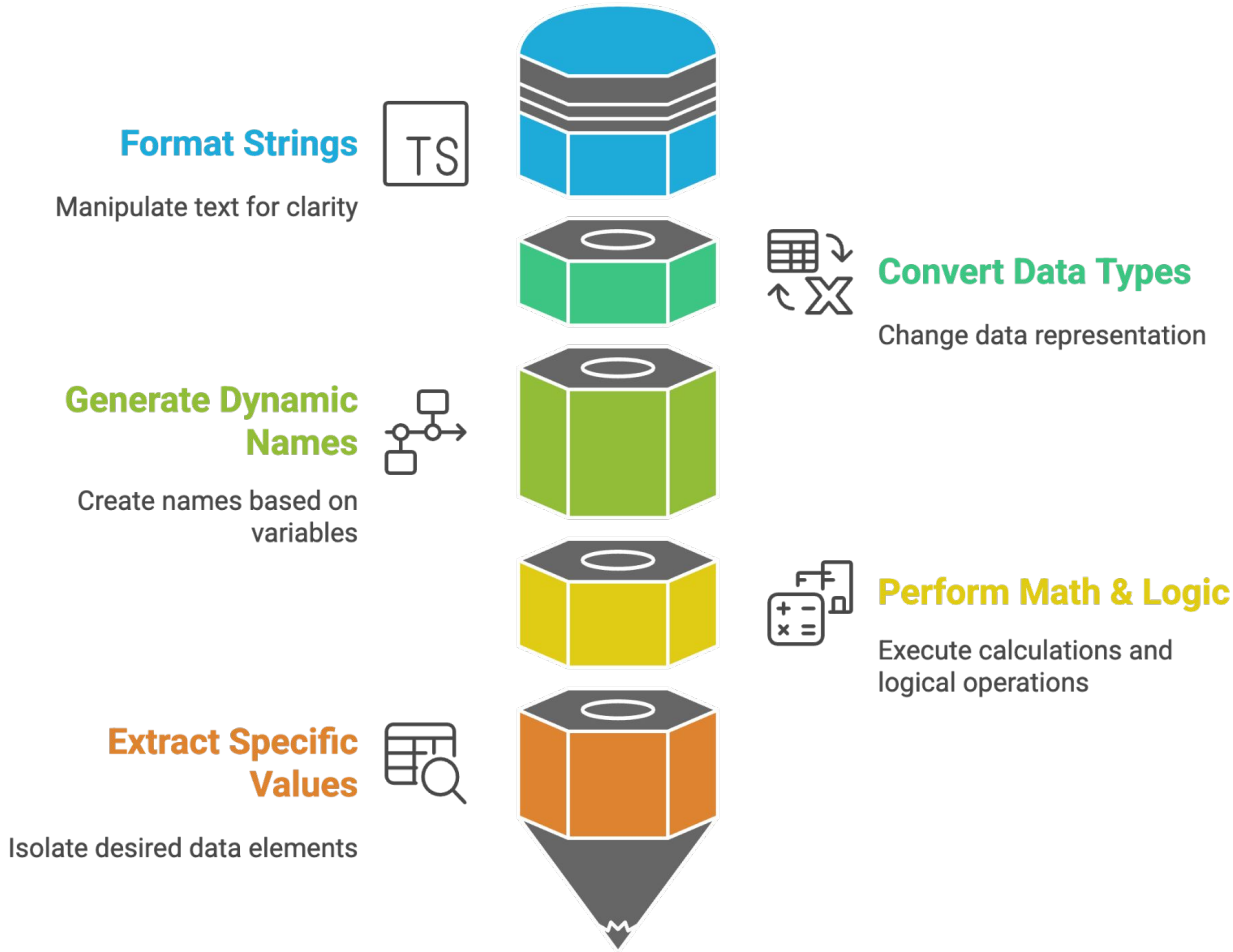
Terraform

Functions

- Terraform functions allow you to **transform**, **combine**, or **compute** values dynamically.
- They help make configurations **more flexible**, **clean**, and **automation-friendly**.

Syntax:

```
function_name(argument1, argument2)
```


Format Strings
TS
Manipulate text for clarity
Convert Data Types
Change data representation
Generate Dynamic Names
Create names based on variables
Perform Math & Logic
Execute calculations and logical operations
Extract Specific Values
Isolate desired data elements

Function Type	Example	Purpose
String	upper(), format()	Modify/format text
Numeric	ceil(), min()	Math operations
Collection	length(), merge()	Work with lists/maps
Filesystem	file()	Read files into variables
Date/Time	timestamp()	Get UTC time
Type Conversion	toset(), tolist()	Convert data types

Functions HandsOn - I

Terraform

Functions HandsOn - II

Terraform

Expressions

Terraform

Expressions

- Expressions are **combinations of variables, functions, operators, and values** that Terraform evaluates to produce a final value.
- They make Terraform configurations **dynamic**, **flexible**, and **automated**.

Where Expressions Are Used

✔ Inside resource arguments

✔ In variables and locals

✔ In outputs

✔ Inside loops (for_each, for expressions)

✔ Inside conditions (<condition> ? <true> : <false>)

Types of Expressions:

Expression Type	Example	Purpose
Literal	"hello", 10, true	Static values
Variable reference	var.instance_type	Use variable value
Resource reference	aws_instance.web.id	Use resource attributes
Function call	upper(var.name)	Transform values
Conditionals	var.env == "dev" ? 1 : 0	Logic-based values
For-expressions	[for s in var.subnets : s.id]	Loops

Usage:

Using Mathematical Expression:

```
count = var.replica_count + 1
```

Using String Expression:

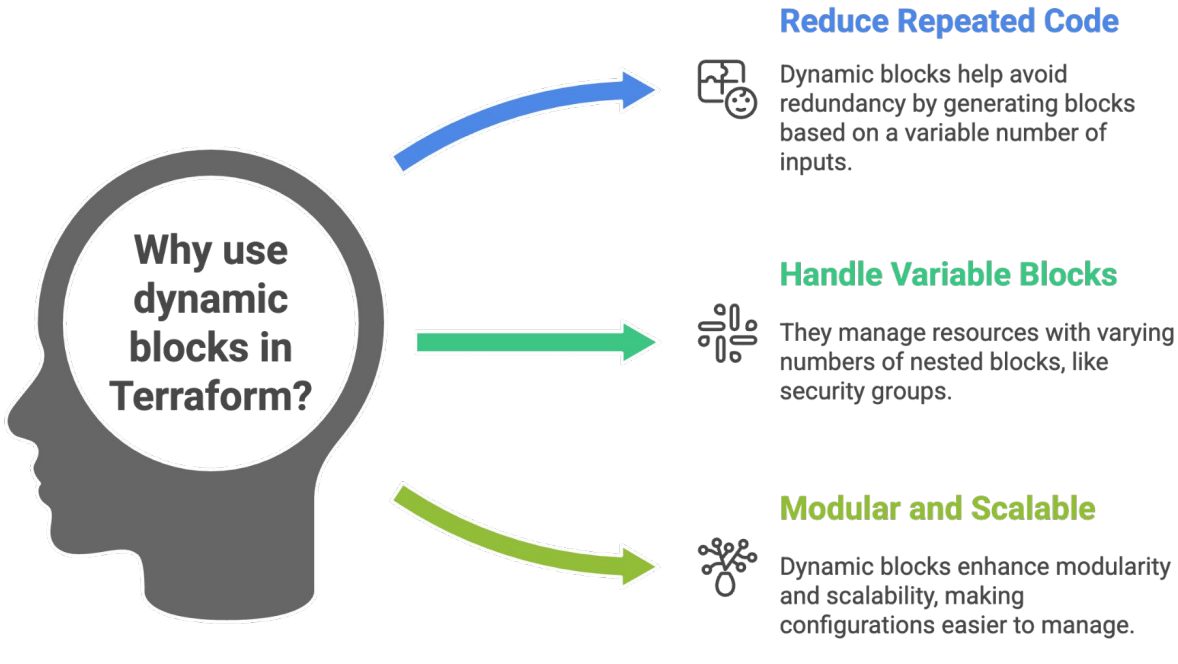
```
tags = {
  Name = upper(var.app_name)
}
```

Dynamic Blocks

Terraform

Dynamic Blocks

- A **dynamic block** allows you to **generate nested blocks dynamically** using loops.
- It is used when the number of nested attributes is **unknown**, **variable**, or **comes from a list/map**.



Syntax:

```
dynamic "block_name" {
  for_each = <collection>

  content {
    # block content using each.key and each.value
  }
}
```

Splat Expressions

Terraform

Splat Expressions

- Splat expressions allow you to **extract attributes from multiple resources** created using count or for_each — without writing loops manually.
- They return a **list of values** (e.g., all IDs, all public IPs).

Types:

Type	Syntax	Description
Legacy Splat	resource.*.attribute	Old style; still widely used
Full Splat	resource[*].attribute	Newer explicit syntax

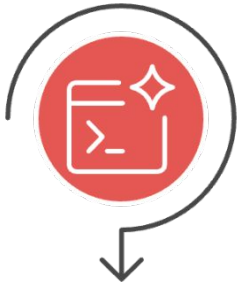
Benefits:

Collect Attributes

Collect attributes from multiple instances.

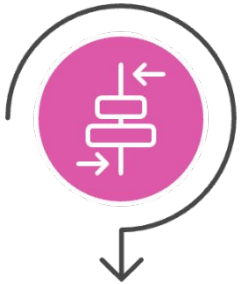
Simplify Definitions

Simplify output definitions in your code.



Clean Code

Make your code clean and more readable.



Useful with Loops

Useful when using `count` or `for_each`.

Usage:

```
resource "aws_instance" "web" {
  count         = 3
  ami           = "ami-123456"
  instance_type = "t2.micro"
}

output "public_ips" {
  value = aws_instance.web[*].public_ip
}
```

Splat Expressions HandsOn

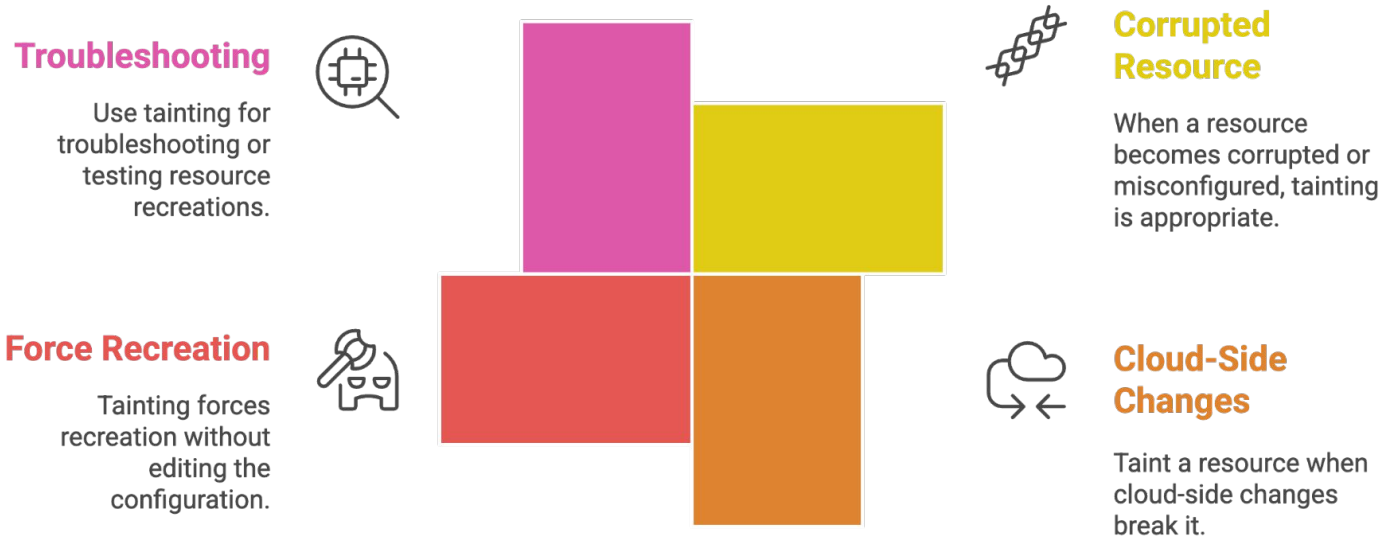
Terraform

Tainting in Terraform

Terraform

Tainting in Terraform

- Tainting is a Terraform operation that **forces a specific resource to be destroyed and recreated** during the next terraform apply.
- It marks a resource as **"tainted"**, meaning Terraform will not try to update it — it will **re-create** it from scratch.



Key Commands:

Command	Purpose
terraform taint <resource>	Mark resource to be recreated
terraform untaint <resource>	Remove taint flag
terraform state show	View resource attributes
terraform apply	Recreates tainted resource

Important Notes:

- Taint does **not** modify the Terraform code
- Only marks **state**
- Resource is recreated **on next apply**, not immediately

Terraform Taint HandsOn

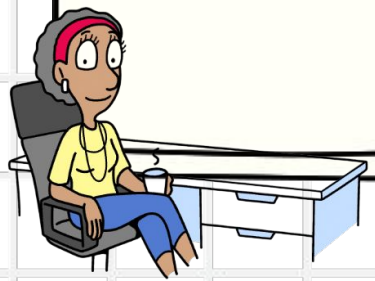
Terraform

Replace HandsOn

Terraform

Zipmap Function

Terraform



Zipmap Functions

- zipmap() is a Terraform function that **combines two lists**

  → one list of **keys**

  → one list of **values**

- It returns a **map** where each key is paired with a corresponding value.

Syntax:

```
zipmap(keys, values)
```

Usage:

```
zipmap(["a", "b", "c"], [1, 2, 3])
```

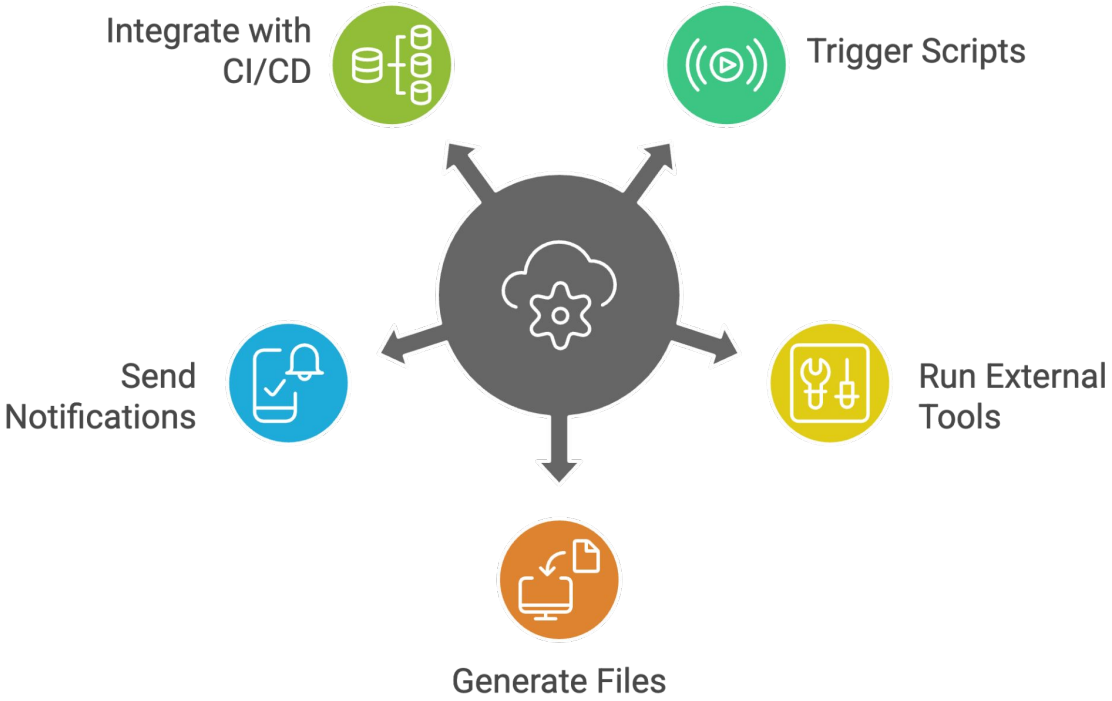
```
{
  a = 1
  b = 2
  c = 3
}
```

Local-Exec

Terraform

Local-Exec

- local-exec is a Terraform provisioner that runs **a command on the machine where Terraform is executed**, *not* on the created remote resource.



Syntax:

```
provisioner "local-exec" {
  command = "your command here"
}
```

Usage:

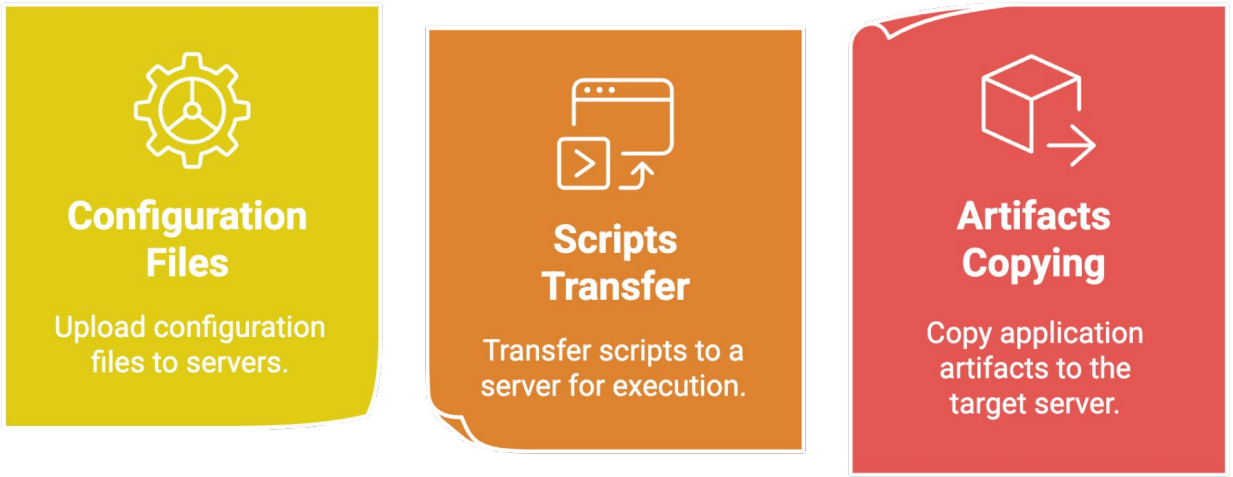
```
resource "aws_instance" "web" {
  ami           = "ami-123456"
  instance_type = "t2.micro"

  provisioner "local-exec" {
    command = "echo Web server created with IP ${self.public_ip} >> created.txt"
  }
}
```

File Provisioner

Terraform

File Provisioner

- The **file provisioner** is used to **copy files or directories** from the **local machine** (where Terraform runs) to a **remote resource** (such as an EC2 instance).
- It performs **file transfer only** — it does **not execute commands**.



Syntax:

```
provisioner "file" {
source      = "local_path"
destination = "remote_path"
}
```

Best Practices:

✔ Copy to writable directories first

✔ Use remote-exec to move or execute files

✔ Avoid large file transfers

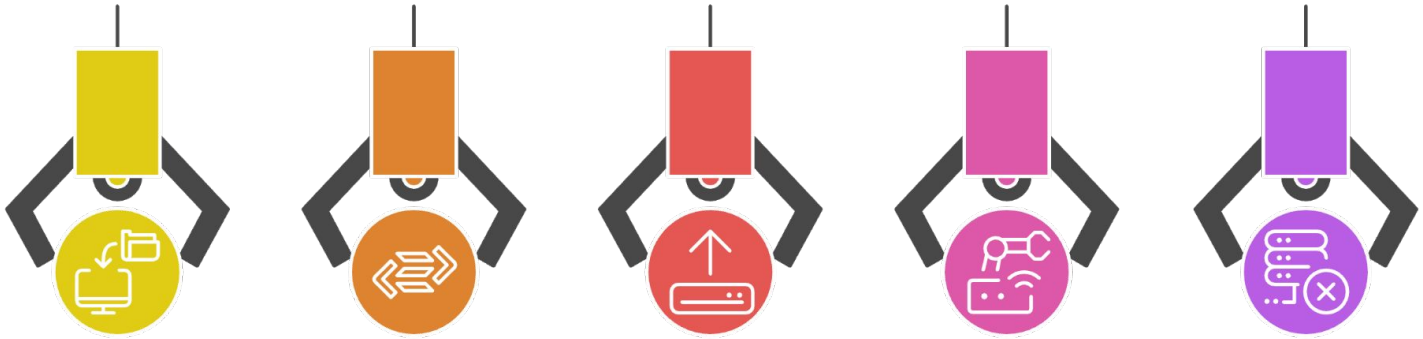
✔ Prefer AMIs or user_data for complex setups

Provisioners

Terraform

Provisioners

- Provisioners allow Terraform to **run scripts or commands** on a local machine or remote resource *after* the resource is created.
- Provisioners are meant for **bootstrapping**, **initial setup**, or **final configuration** when cloud-native solutions are not enough.



Software Installation

Install software on a virtual machine.

Shell Scripts

Run initialization shell scripts on a VM.

File Upload

Upload configuration files to a virtual machine.

Automation Tools

Trigger external automation tools from a VM.

Unsupported Tasks

Perform tasks not directly supported by providers.

Types of Provisioners:

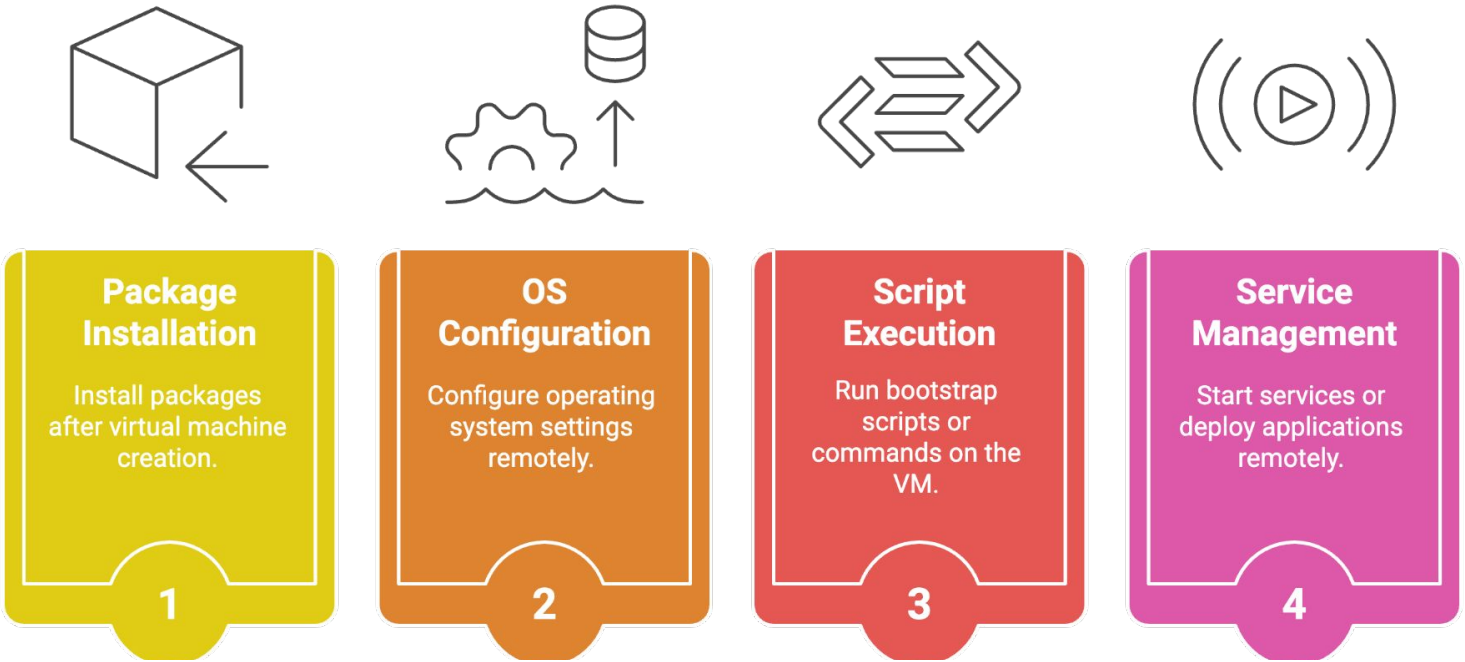
Provisioner	Purpose
local-exec	Runs commands **locally** (where Terraform runs)
remote-exec	Runs commands **on the remote resource** (e.g., EC2)
file	Upload files or directories to a remote machine

Remote-Exec

Terraform

Remote-Exec

- remote-exec is a Terraform provisioner used to **run commands on a remote machine** (such as an EC2 instance) **after** the resource is created.



Syntax:

```
provisioner "remote-exec" {
  inline = [
    "command 1",
    "command 2",
  ]
}
```

Usage:

```
resource "aws_instance" "web" {
  ami           = "ami-123456"
  instance_type = "t2.micro"

  # Remote exec runs on the created EC2 instance
  provisioner "remote-exec" {
    inline = [
      "sudo apt update -y",
      "sudo apt install -y nginx",
      "sudo systemctl enable nginx"
    ]
  }

  connection {
    type        = "ssh"
    user        = "ubuntu"
    host        = self.public_ip
    private_key = file("~/.ssh/id_rsa")
  }
}
```

Resources

Terraform

Resources

- A **resource** is the **most important building block** in Terraform — it represents a real-world infrastructure component such as:
  a. A virtual machine (EC2, VM)
  b. A network (VPC, subnet)
  c. A database, S3 bucket, load balancer, etc.

Purpose:

- Define what infrastructure Terraform should create or manage.
- Describe the desired **state** of each infrastructure object.
- Enable **repeatable and version-controlled** infrastructure deployment.

Examples of Resources:

- aws_instance → EC2 instance
- azurerm_resource_group → Azure Resource Group
- google_storage_bucket → GCP bucket

Syntax:

```
resource "<PROVIDER>_<RESOURCE_TYPE>" "<NAME>" {
  argument1 = "value"
  argument2 = "value"
}
```

Sample:

```
resource "aws_instance" "web_server" {
  ami           = "ami-0abcd1234"
  instance_type = "t2.micro"

  tags = {
    Name = "MyWebServer"
  }
}
```

Part	Meaning
resource	Keyword to declare a resource
"aws_instance"	Resource type (from provider)
"web_server"	Local name (identifier within Terraform)
{ ... }	Configuration arguments (e.g., AMI, instance type)

Authentication Methods

AWS

Terraform

Hardcoding Credentials

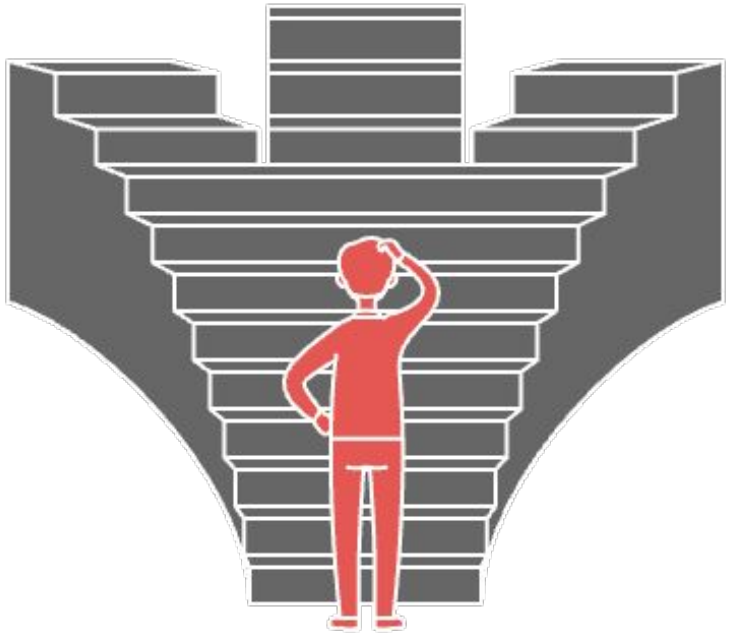
Exposes sensitive data directly in the code, leading to security vulnerabilities.

Using Environment Variables

Stores credentials outside the codebase, enhancing security.

Utilizing Credentials File

Provides a structured way to manage sensitive information, keeping it separate from application logic.



State Backup File

Terraform

State Backup File

- A **state backup file** is an automatically created copy of the previous Terraform state, stored locally as terraform.tfstate.backup.

Example Scenario:

- You run terraform apply → Terraform updates infrastructure.
- Before saving the new state, it creates a **backup** of the old one.
- If something goes wrong, you can restore the backup:

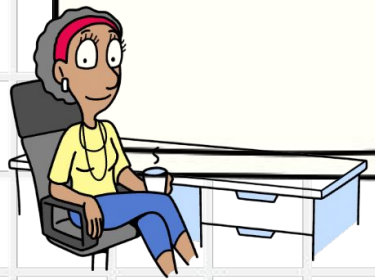
```
mv terraform.tfstate.backup terraform.tfstate
```

Best Practices:

- Always **version-control** or store backups securely (not public repos).
- For teams, use **remote state with versioning** (e.g., S3 with versioning enabled).
- Never manually edit the .tfstate or .backup file unless absolutely necessary.

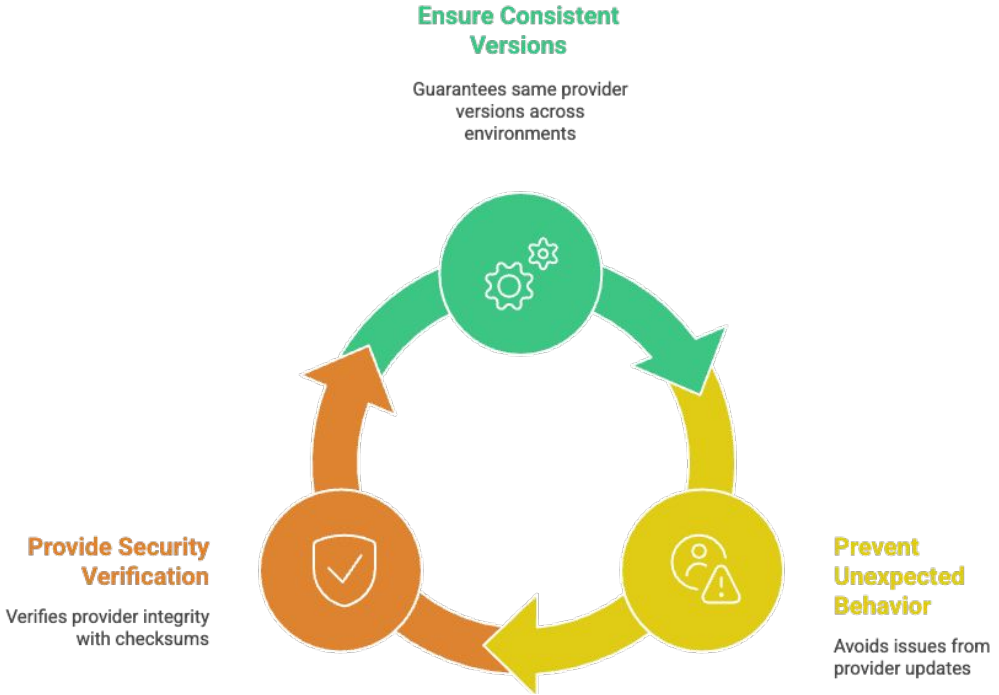
Dependency Lock File

Terraform



Dependency Lock File

- A **Dependency Lock File** (.terraform.lock.hcl) is an automatically generated file that **records the exact versions and checksums** of provider plugins used in your Terraform project.

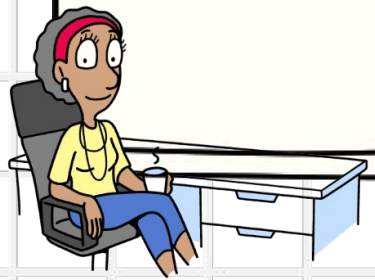


Key Benefit:

Aspect	Description
Version Pinning	Locks providers to specific versions.
Hash Verification	Ensures downloaded providers are authentic and unmodified.
Automatic Update	Updated only when provider versions change.
Team Consistency	Keeps all environments aligned on same provider versions.

Remote State File

Terraform



Remote State File

- A *Remote State File* stores Terraform's state (terraform.tfstate) in a shared, remote location instead of locally.

Common Remote Backends:

- **AWS S3** (most used)
- **Azure Blob Storage**
- **Google Cloud Storage (GCS)**
- **Terraform Cloud / Enterprise**

Key Benefit:

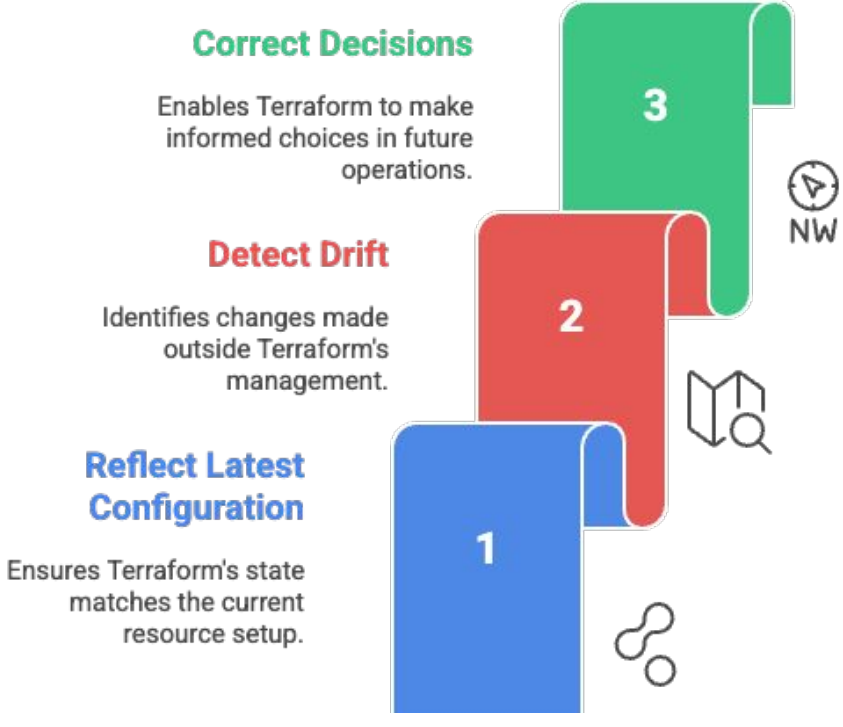
- Centralized, secure, and version-controlled infrastructure state management.

Terraform Refresh

Terraform

Terraform Refresh

- terraform refresh is a command that **updates the state file** (terraform.tfstate) to match the **current real-world infrastructure**.



Example Scenario:

- You manually change an EC2 instance tag in AWS Console.
- Terraform's state is now **out of sync** with real AWS.
- Run:

```
terraform refresh
```

→ Terraform updates its local state file to match AWS's actual configuration.

Best Practices:

- Run terraform refresh or terraform plan before apply to detect drift.
- Avoid frequent manual infrastructure changes — use Terraform for consistency.
- Review plan outputs after refresh to understand detected drift.
- terraform refresh **does not create or destroy** any resource — it only updates the **state file** to represent reality.

Terraform Refresh HandsOn

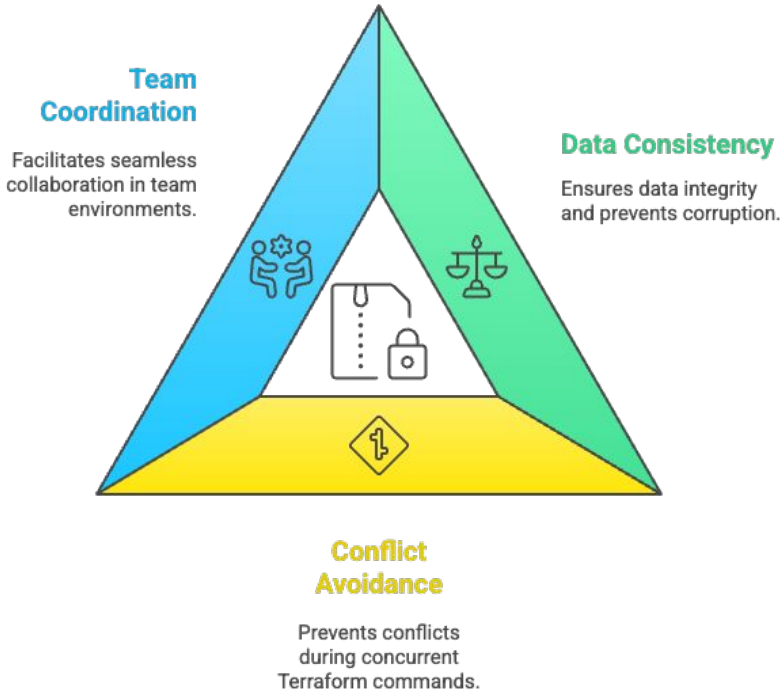
Terraform

Lock State File

Terraform

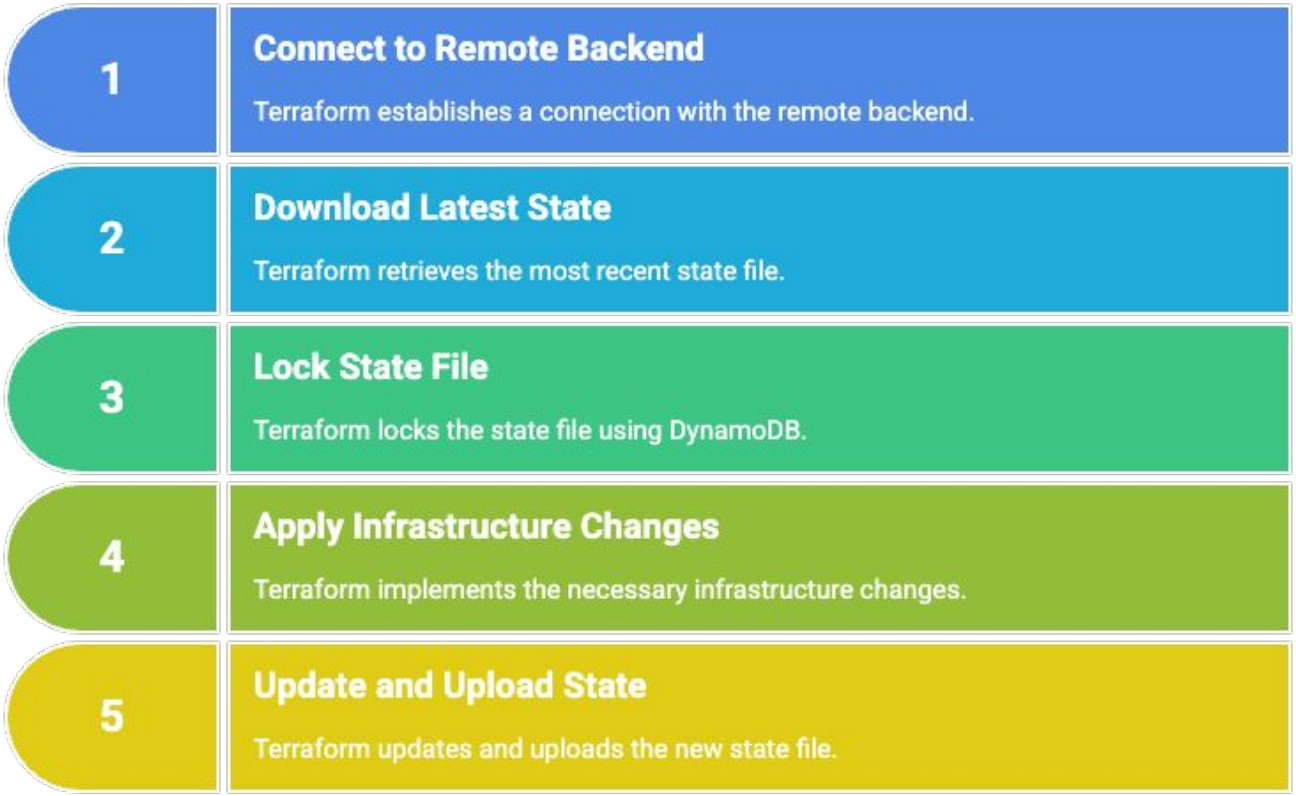
Lock State File

- A **state lock file** is a mechanism that prevents multiple users or processes from modifying the same Terraform state file at the same time.



```hcl
terraform {
  backend "s3" {
    bucket         = "my-terraform-state-bucket"
    key            = "prod/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "terraform-lock"
    encrypt        = true
  }
}
```

Terraform Workflow with Remote Backend

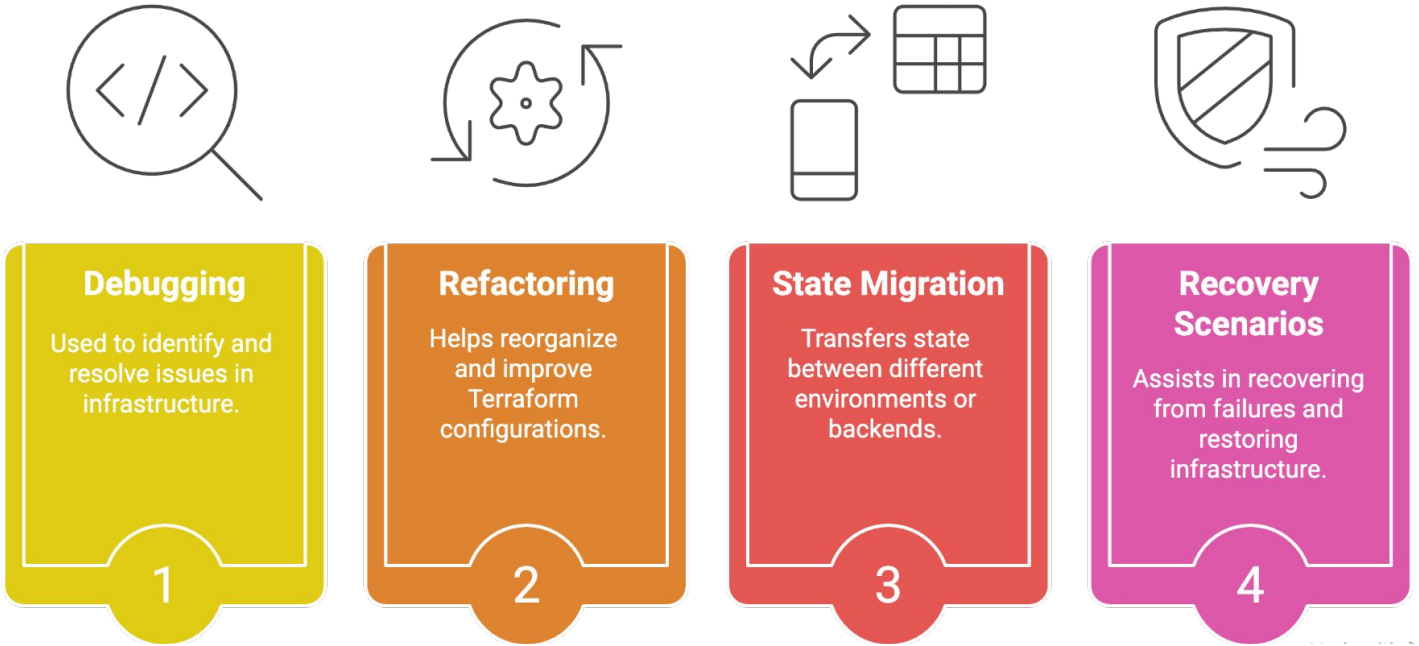


State File Commands

Terraform

State File Commands

- Terraform **state commands** allow you to **inspect, modify, and manage the state file** safely without changing real infrastructure directly.



Command	Purpose
terraform state list	List all resources in state
terraform state show	Show details of a resource
terraform state mv	Move/rename resources in state
terraform state rm	Remove resource from state
terraform state pull	Download remote state
terraform state push	Upload state manually (advanced)

Types of IaC

Terraform

Types of IaC

- Infrastructure as Code is the practice of **managing and provisioning infrastructure using code** instead of manual processes.
- **Terraform for provisioning + Ansible for configuration**
  This combination gives **clean separation of responsibilities** and scalable automation.

Types of IaC (Purpose Based):

Category	Purpose	Examples
Provisioning	Create cloud resources	Terraform, CloudFormation
Configuration Management	Configure OS & apps	Ansible, Chef, Puppet

IaC Types Comparison (Approach Based)

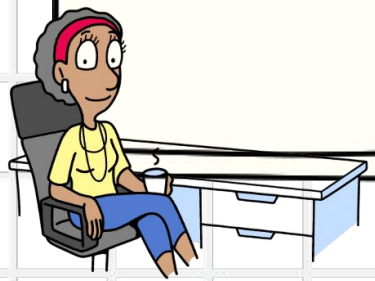
Characteristic	Declarative IaC	Imperative IaC
Definition	Defines final state	Defines step-by-step instructions
Tool Control	Tool decides how to achieve it	You control how infrastructure is created
Safety	Safer and more predictable	Less safe and predictable
Examples	Terraform, CloudFormation, ARM Templates	Shell scripts, AWS CLI scripts, Pulumi

Comparison:

Feature	Terraform	Ansible	Chef / Puppet	CloudFormation
Primary Role	Provisioning	Configuration	Configuration	Provisioning
State Management	Yes (tfstate)	No	Yes (server)	Managed by AWS
Multi-Cloud	✅ Yes	⚠️ Partial	❌ No	❌ AWS only
Drift Detection	Yes	No	Limited	Yes
Best Use Case	Cloud infra lifecycle	OS & app config	Large fleets	AWS-only infra

Null Resource

Terraform



Null Resource

null_resource is a **special Terraform resource** that:

- Does **not create any real infrastructure**
- Exists only to **run provisioners or orchestration logic**
- Is commonly used for **scripts, automation, and glue logic**

Key Feature: triggers

- triggers define **when the null_resource should re-run**
- If trigger values change → resource is recreated → provisioner runs again

Null Resource Use Cases

Provisioner Execution

Run provisioners without creating cloud resources.

Script Execution

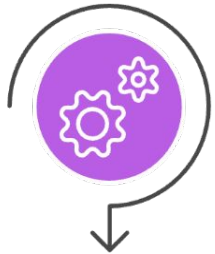
Execute scripts on already created servers.

Execution Order

Control the execution order of provisioners.

Change Trigger

Trigger actions based on changes in configuration.



Integration

Integrate Terraform with CI/CD or external tools.

Syntax:

```
resource "null_resource" "example" {
  triggers = {
    key = "value"
  }
}
```

Usage:

```
resource "null_resource" "configure" {
  triggers = {
    instance_id = aws_instance.web.id
  }
  provisioner "remote-exec" {
    inline = [
      "sudo apt update",
      "sudo apt install -y nginx"
    ]
  }
  connection {
    type        = "ssh"
    user        = "ubuntu"
    host        = aws_instance.web.public_ip
    private_key = file("~/.ssh/id_rsa")
  }
}
```

Terraform Logs

Terraform

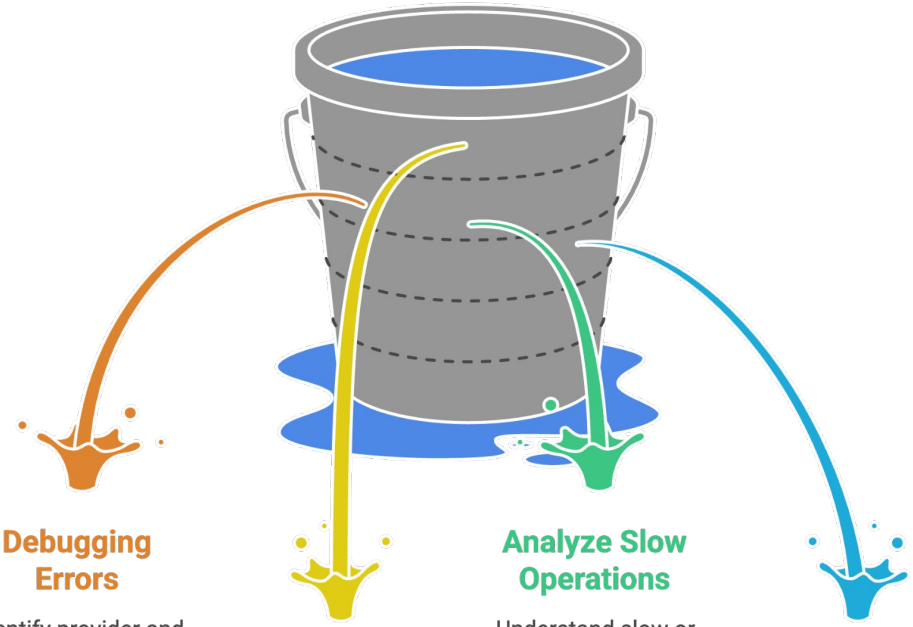
Terraform Logs

Terraform logs provide **detailed internal execution information** that helps you:

- Debug errors
- Understand provider behavior
- Analyze API calls
- Troubleshoot authentication and state issues

NOTE: Terraform logging is controlled using **environment variables**.

Understanding Terraform Logs



Debugging Errors

Identify provider and auth issues

Investigate Failures

Analyze failed apply or plan

Analyze Slow Operations

Understand slow or stuck operations

Internal Understanding

See what Terraform is doing

Log Levels:

Level	Description
TRACE	Most detailed (API calls, internals)
DEBUG	Debugging info (provider actions)
INFO	High-level operations
WARN	Warnings
ERROR	Errors only

Syntax:

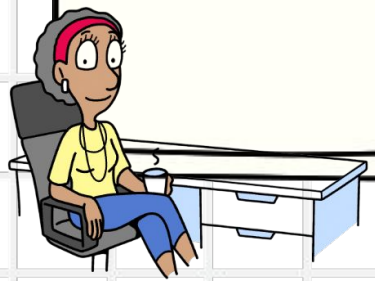
```
karangupta@KaranGupta terraform-test %
karangupta@KaranGupta terraform-test % export TF_LOG=DEBUG
karangupta@KaranGupta terraform-test %
```

Terraform Logs HandsOn

Terraform

Terraform Workspaces

Terraform



Terraform Workspaces

- Terraform workspaces allow you to **manage multiple environments** (such as dev, test, prod) using the **same Terraform configuration**.

  Each workspace has:

  - A **separate state file**
  - The **same codebase**
  - Different infrastructure instances

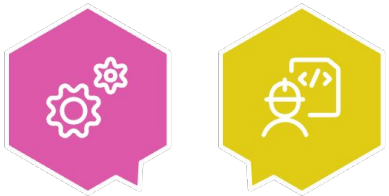
Syntax:

```
terraform workspace list
terraform workspace show
terraform workspace new dev
terraform workspace select prod
```

Workspace Benefits

Simple Setups

Useful for simple multi-environment setups, streamlining workflows.

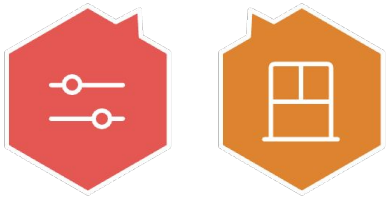


Code Reuse

Avoid duplicating Terraform code, promoting efficiency.

Quick Switching

Quickly switch between environments for easy management.



Environment Isolation

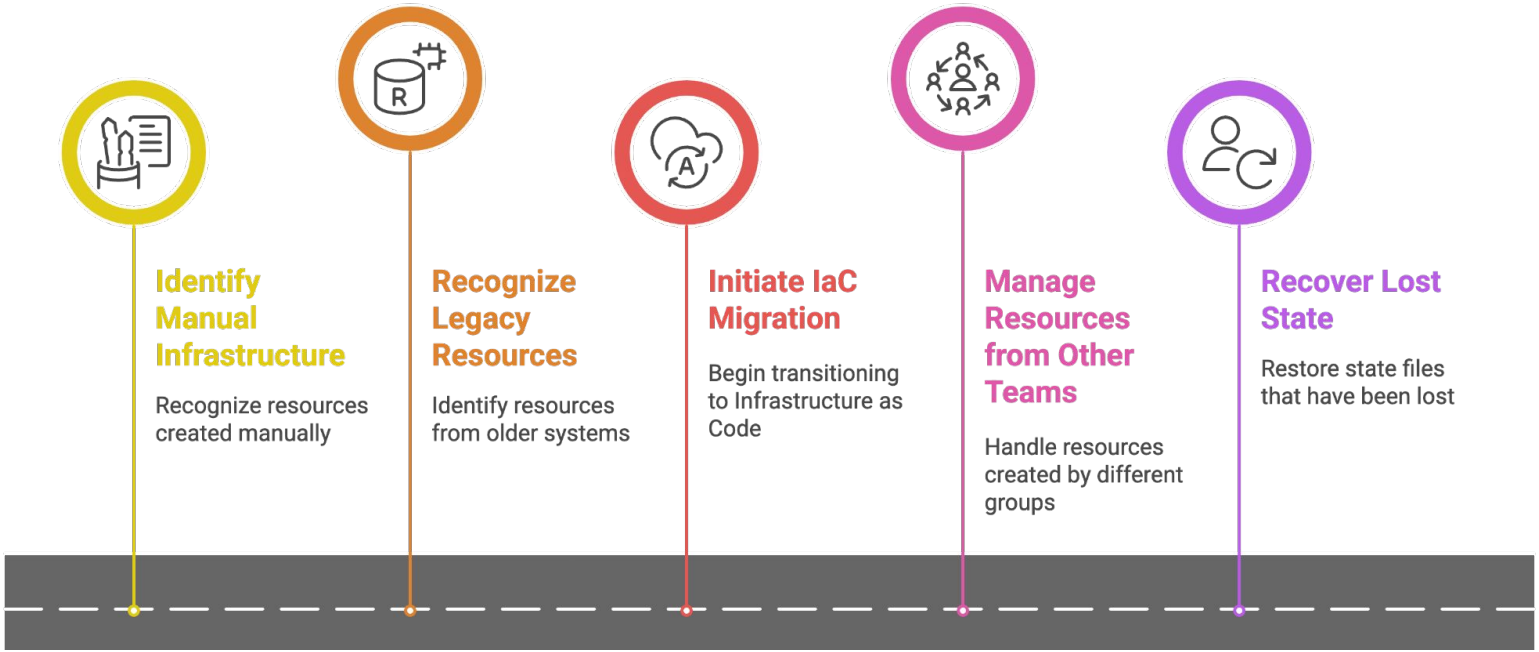
Isolate environments using separate state files for clarity.

Terraform Import

Terraform

Terraform Import

- terraform import is used to **bring existing infrastructure under Terraform management without recreating it**.

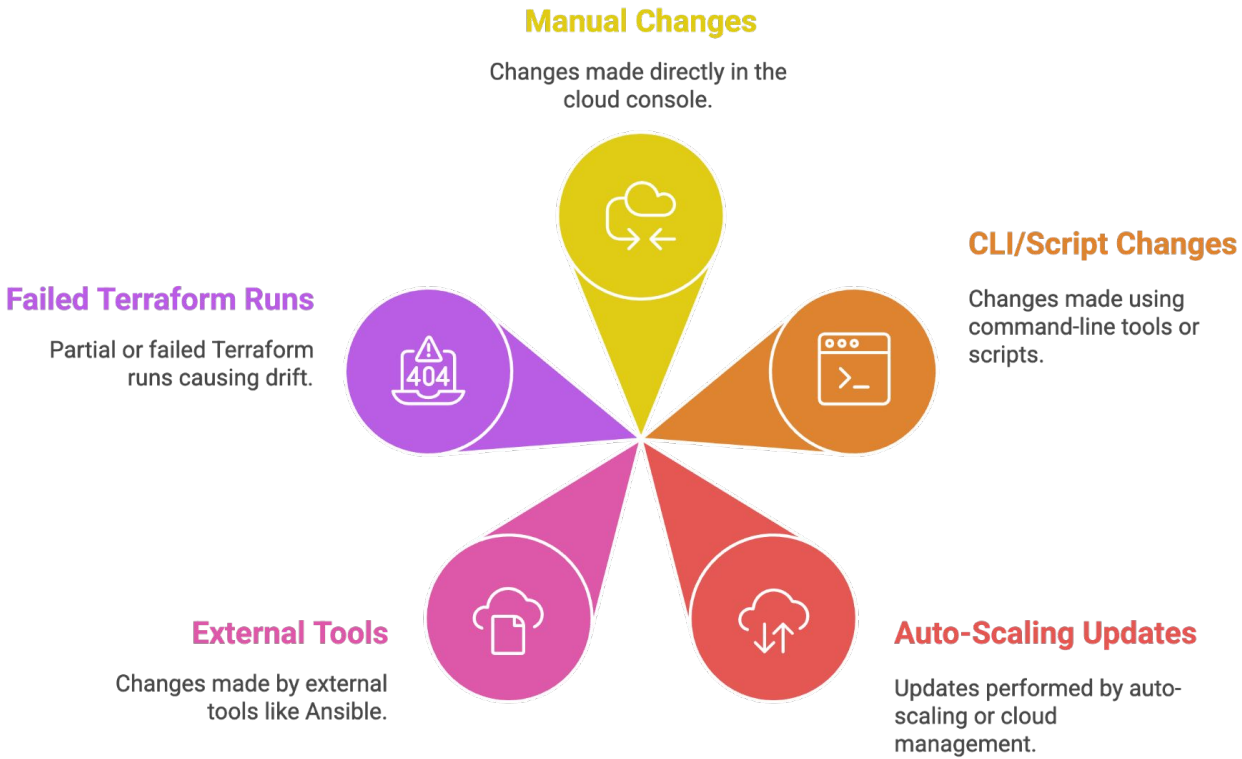


Drift Changes

Terraform

Drift Changes

- **Drift** occurs when the **actual infrastructure** changes **outside Terraform**, but Terraform's **state file is not updated**.



Why Drift Is Dangerous:

⚠️ Unexpected changes during terraform apply

⚠️ Infrastructure inconsistency

⚠️ Security & compliance risks

⚠️ Production downtime risk

Best Practices:

✔ Avoid manual changes outside Terraform

✔ Use terraform plan regularly

✔ Enable drift detection in CI/CD

✔ Use ignore_changes carefully

✔ Treat Terraform as the **single source of truth**

Drift Changes HandsOn

Terraform

Ignore Changes HandsOn

Terraform

Terraform Graph

Terraform

Terraform Graph

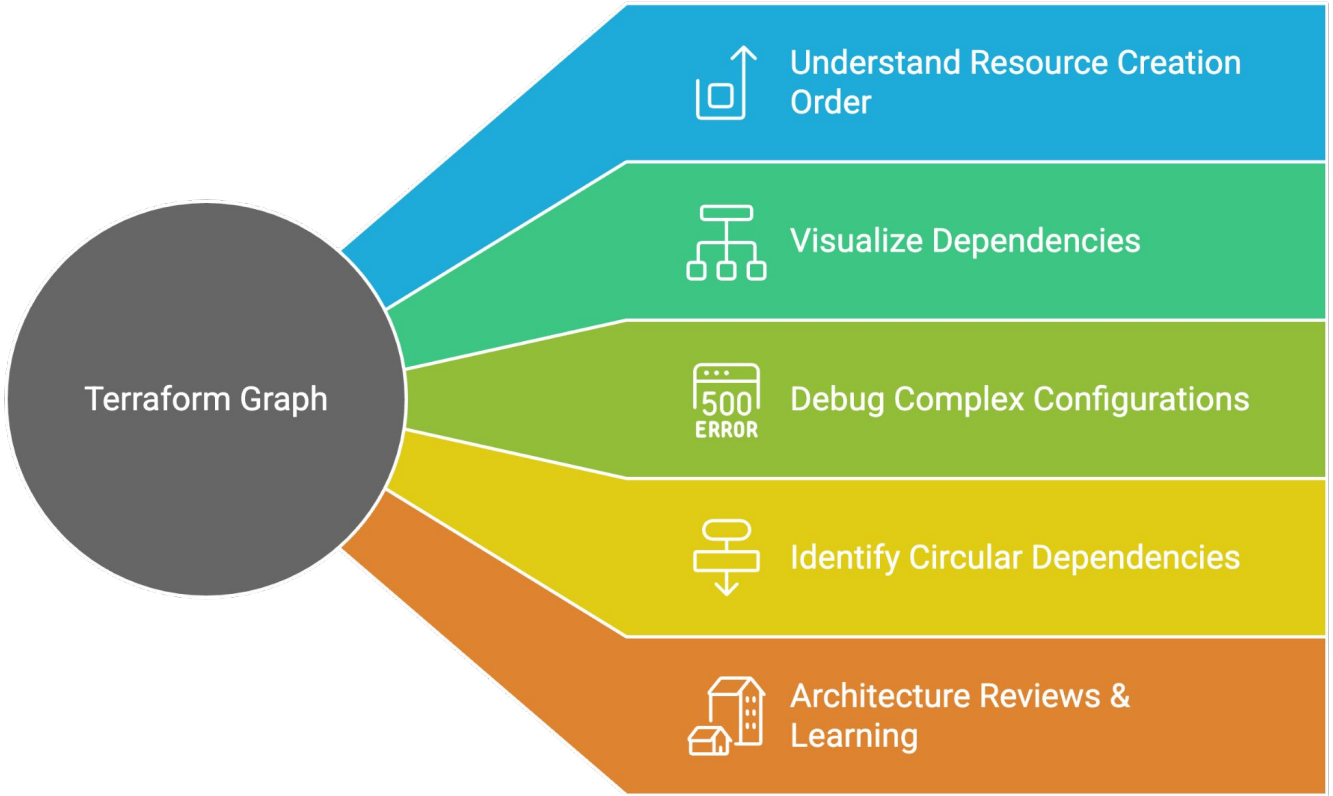
- terraform graph is a CLI command that **visualizes the dependency relationships** between resources.
- It generates a **dependency graph** that shows:
  - ➔ Resources
  - ➔ Data sources
  - ➔ Providers
  - ➔ How they depend on each other

Syntax:

```
terraform graph
```

```
sudo apt install graphviz
terraform graph | dot -Tpng > terraform-graph.png
```

Unveiling Terraform Graph's Benefits



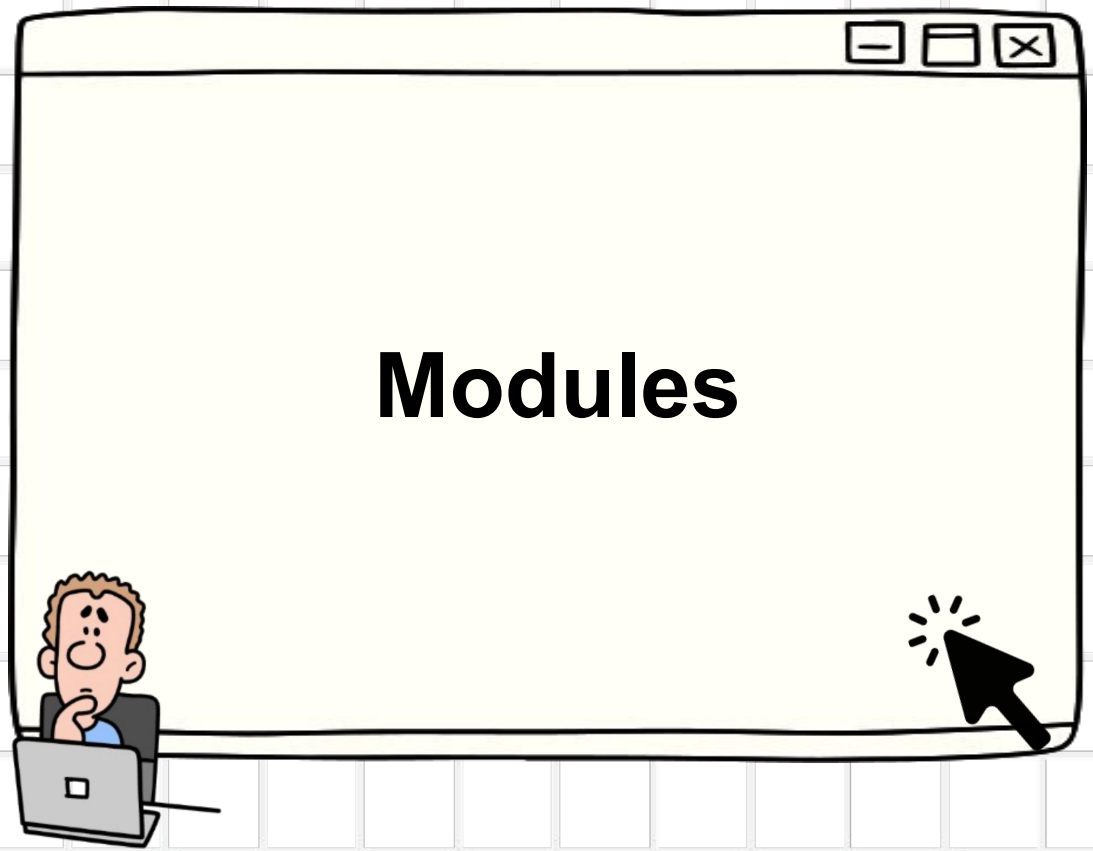
Fetching AMI

HandsOn

Terraform

Modules

Terraform

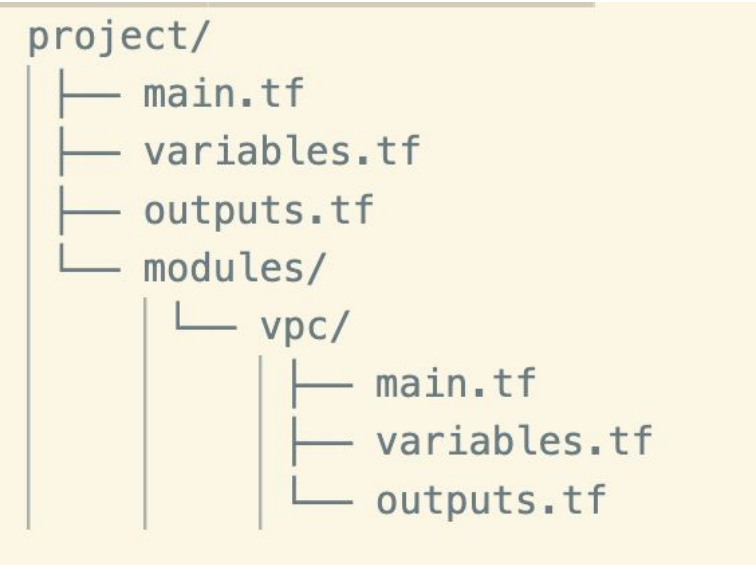


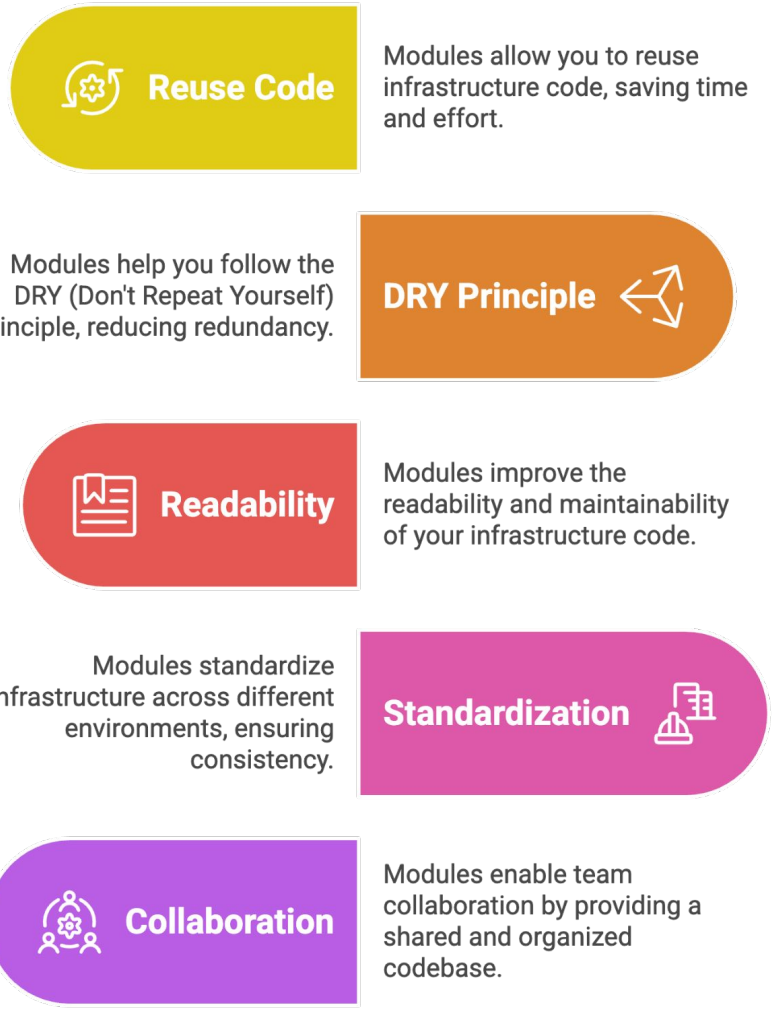
Modules

- A **Terraform module** is a **set of Terraform configuration files** grouped together to perform a **specific infrastructure task**.
- Terraform modules help you **organize, reuse, and scale infrastructure code** efficiently.

Note:

- Every Terraform configuration is a module.
- The main working directory is called the **Root Module**.




Reuse Code
Modules allow you to reuse infrastructure code, saving time and effort.
Modules help you follow the DRY (Don't Repeat Yourself) principle, reducing redundancy.
DRY Principle
Readability
Modules improve the readability and maintainability of your infrastructure code.
Modules standardize infrastructure across different environments, ensuring consistency.
Standardization
Collaboration
Modules enable team collaboration by providing a shared and organized codebase.

Module Type	Description
Root Module	Main directory where terraform apply runs
Local Module	Module stored in local filesystem
Remote Module	Module stored in Git, S3, or registry
Registry Module	Prebuilt modules from Terraform Registry

Best Practices:

✔ One module = one responsibility

✔ Always define variables and outputs

✔ Avoid hardcoded values inside modules

Input Validation

Terraform


Validation
Avoid Runtime Failures
Improve User Experience
Enforce Naming Conventions
Enforce Security and Compliance Rules
Catch Mistakes Early

Terraform Cloud

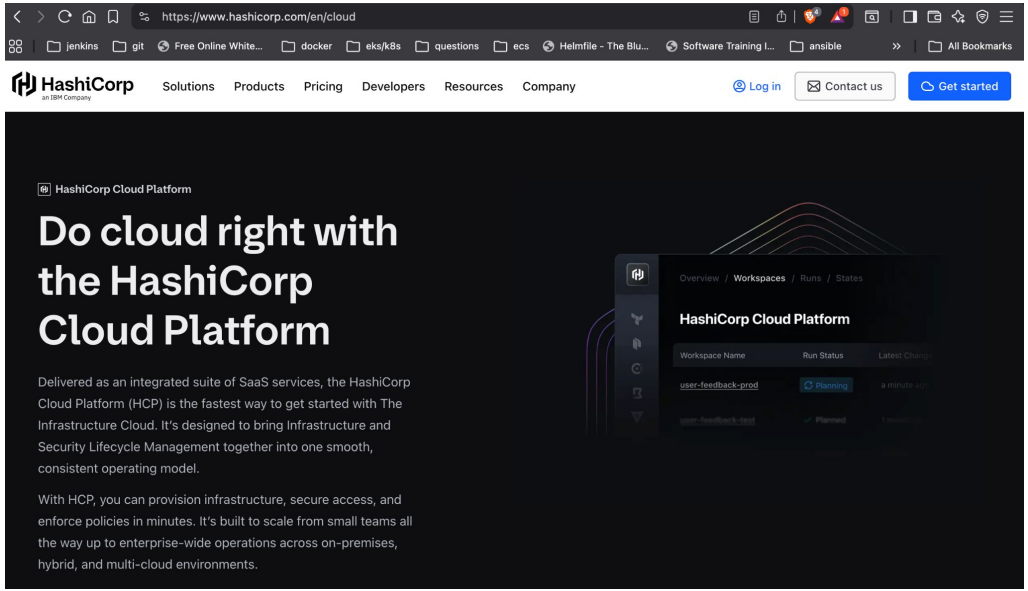
Terraform

Terraform Cloud

- Terraform Cloud is a **fully managed SaaS platform** by HashiCorp that enables teams to run Terraform in a **remote, collaborative environment**.

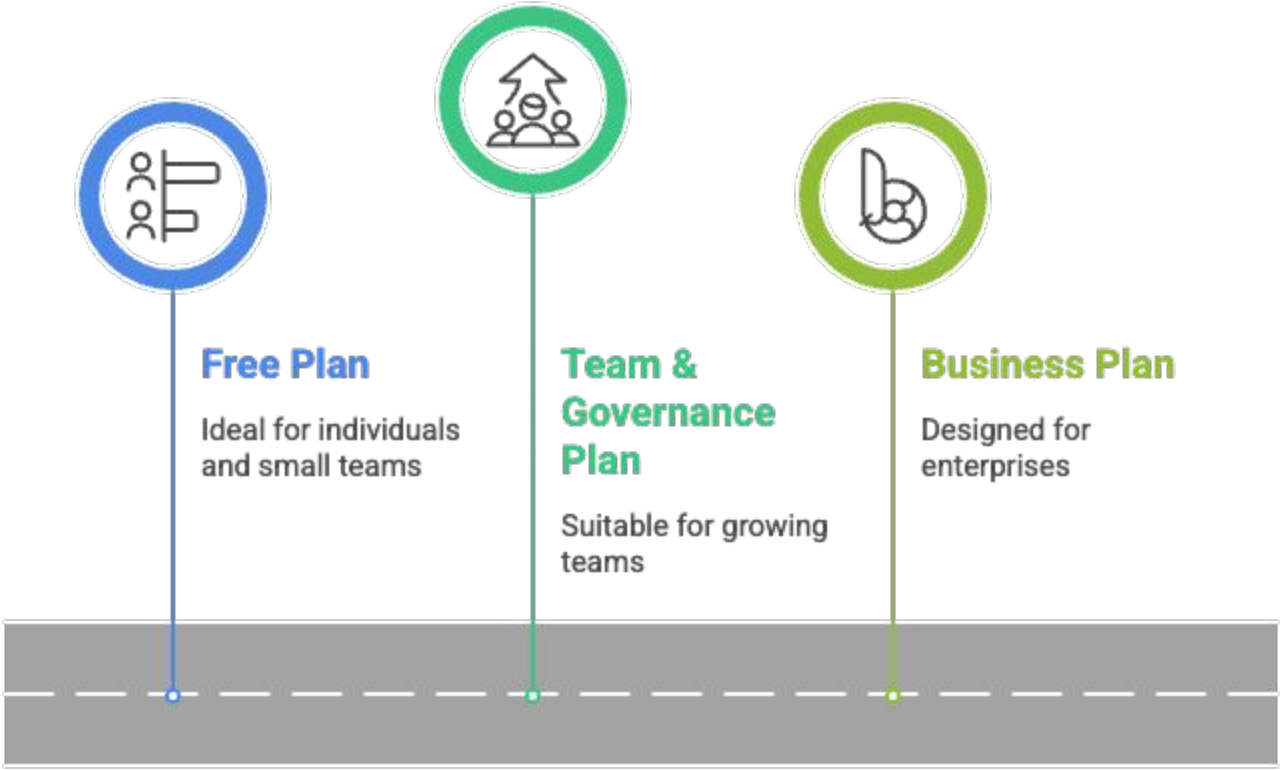
<u>It removes the need to manage:</u>

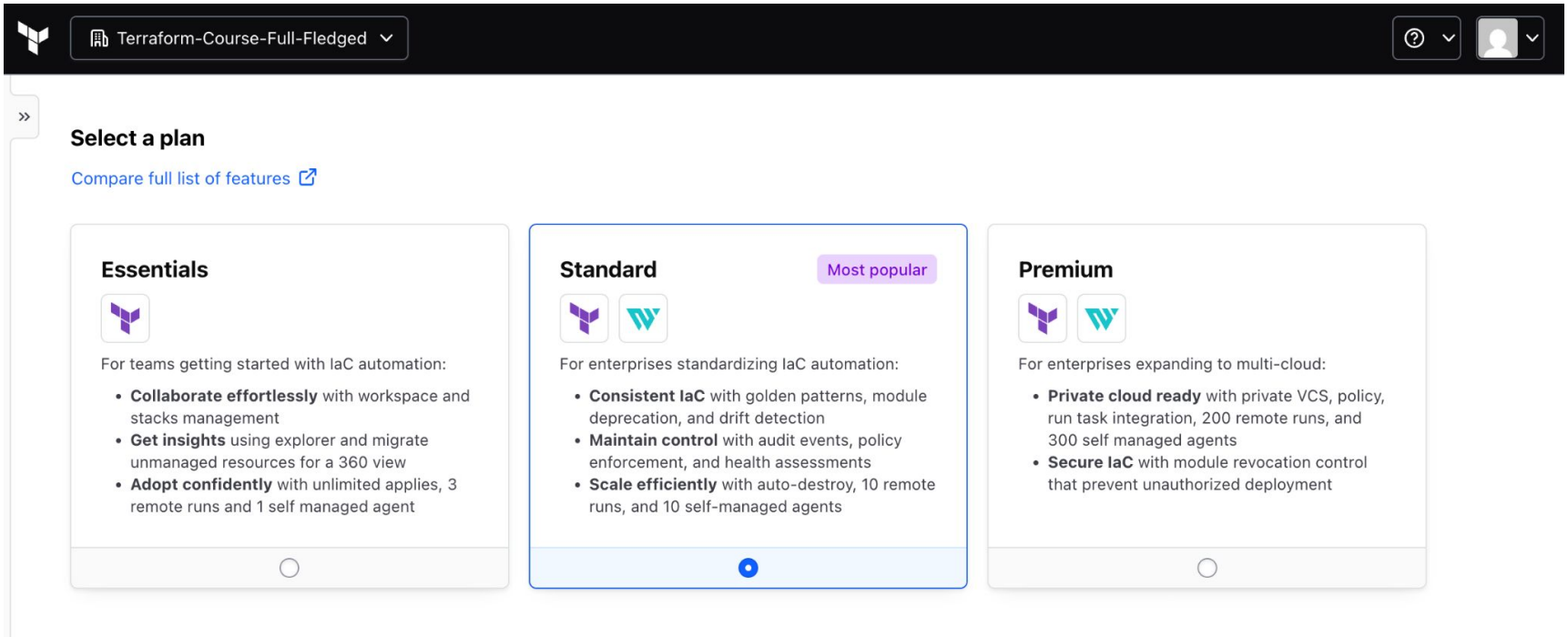
- Local state files
- Backend infrastructure
- Manual state locking



Terraform Cloud Features

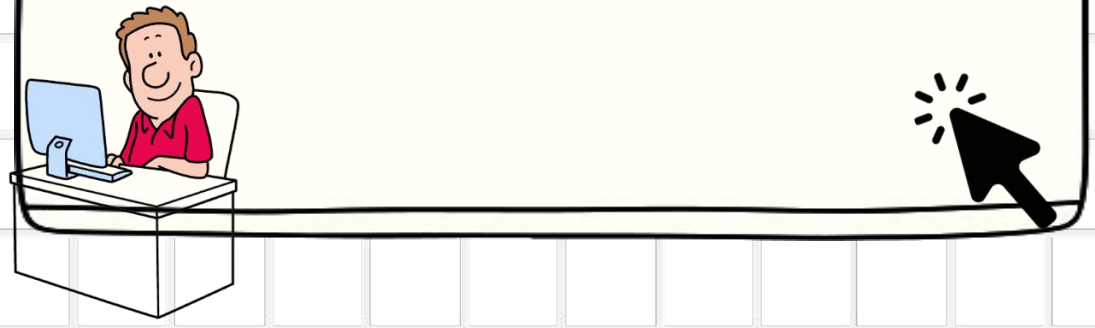
Terraform Cloud Plan Progression




Terraform-Course-Full-Fledged
Select a plan
Compare full list of features
Essentials
For teams getting started with IaC automation:
Collaborate effortlessly with workspace and stacks management
Get insights using explorer and migrate unmanaged resources for a 360 view
Adopt confidently with unlimited applies, 3 remote runs and 1 self managed agent
Standard
Most popular
For enterprises standardizing IaC automation:
Consistent IaC with golden patterns, module deprecation, and drift detection
Maintain control with audit events, policy enforcement, and health assessments
Scale efficiently with auto-destroy, 10 remote runs, and 10 self-managed agents
Premium
For enterprises expanding to multi-cloud:
Private cloud ready with private VCS, policy, run task integration, 200 remote runs, and 300 self managed agents
Secure IaC with module revocation control that prevent unauthorized deployment

Sentinel

Terraform



Sentinel

- **Sentinel** is a **Policy as Code framework** developed by HashiCorp and integrated into Terraform Cloud.
- It allows organizations to **define and enforce rules** before infrastructure changes are applied.

How Sentinel Works:

1. Terraform generates a plan
2. Sentinel evaluates policy against the plan
3. If compliant → Apply allowed
4. If not compliant → Run blocked